

Private Verification of Action Eligibility across Trust Domains

AI Safety: Evidence Semantics and Controlled Execution for Cross Organisation Agents

Carsten Stöcker

28 September 2026



Except where otherwise noted, this work is licensed under the Creative Commons Attribution 4.0 International License (CC BY 4.0). © 2026 Dr. Carsten Stöcker, Spherity GmbH.

Abstract

An AI agent can present an authentic claim while lacking current authority, adequate deployment evidence or a safe continuation for its next action. Private verification makes this distinction harder when a relying organisation cannot inspect the underlying evidence. We specify a bounded protocol that connects private policy evaluation to typed claim validation and protected execution across trust domains. A decision binds an exact operation and continuation to a versioned evidence view, including required sources of prohibitions and adverse findings. Claim-specific recognition rules preserve issuer scope through delegation, semantic mapping and explicitly justified composition. A pure deterministic evaluator is separated from a serialised admission controller and resource-specific effect adapters. The controller checks current applicability, consumes a durable logical operation identifier and retains pending effects through cancellation, renewal and agent replacement. Each newly issued continuation permanently charges one coherent episode account. We give conditional proofs of evaluation fidelity, scope preservation, applicable admission and accounting integrity, and state the additional obligations for implementation refinement and probability preservation. Exhaustive exploration of a finite control abstraction reaches 16,020 states and 60,238 transitions; all nine invariants hold, while eight omitted controls produce counterexamples. A proposed Agent2Agent extension uses verifiable credentials and a bounded Open Digital Rights Language profile without attributing assurance semantics to transport conformance. A separate plaintext reference gate passes 29 functional scenarios. A documentation-based OpenClaw case, NVIDIA architecture and published OpenAI and Anthropic practices connect the model to engineering controls and scoped assessment evidence. The result is a protocol specification, formal control argument and reproducible reference implementation of selected functions. It does not establish a deployed private-proof implementation, measured operational safety probabilities or general AI alignment.

Keywords: AI safety; private verification; Agent2Agent; verifiable credentials; ODRL; evidence graphs; scoped trust; claim validation; runtime assurance; cross organisation verification

Contents

1	Introduction	4
2	Related work and standards baseline	5
2.1	Authorisation proofs, confidential credentials and consumable authority	5
2.2	Semantic evidence and claim validation	6
2.3	Feasibility over current protocol foundations	7
2.4	Standards development and contribution boundary	8
3	Evidence semantics and cross domain claim validation	8
3.1	Claims as bounded propositions	8
3.2	Four valued validation	10
3.3	Semantic representation and the bounded policy profile	10
3.4	Scoped recognition across trust domains	12
4	Formal model and conditional properties	12
4.1	Formal model and security boundary	12
4.2	Bounded evidence and claim semantics	14
4.3	Private evaluation and scope preservation	16
4.4	Authoritative admission and continuation state	17
4.5	Accounting, pending effects and operational conformance	19
4.6	Refinement, stochastic transfer and privacy	20
5	Protocol and protected execution	22
5.1	Negotiation and operation binding	22
5.2	Evaluation and readiness	22
5.3	Admission and dispatch	23
5.4	Revocation and the containment protocol	25
5.5	Record types and failure reporting	26
6	Executable validation	27
6.1	Model and method	27
6.2	Baseline and omitted controls	27
6.3	Further counterexamples and validation limits	28
6.4	A separate executable reference gate	29
7	Evidence mechanisms and practical assessment	29
7.1	From an assessment artifact to a usable claim	29
7.2	Evidence produced by existing mechanisms	30
7.3	Assessment provenance and recognised competence	31
7.4	Industrial illustration and developments in assessment practice	32
8	OpenClaw case study: protected engineering-data release	33
8.1	Why OpenClaw is a useful design case	33
8.2	A protected engineering-data workflow	33
8.3	Adversarial traces and the paper’s results	35
8.4	Refinement obligations and validation status	36
9	Industry architectures and published safety practices	37
9.1	NVIDIA: architecture, enforcement and evidence	37
9.2	Evidence status of provider practices	39
9.3	OpenAI: action boundaries and evidence	39
9.4	Anthropic: containment, consent and classifiers	40
9.5	Governance and independent assessment	41
10	A working framework for AI safety engineers	42
10.1	Convert the safety argument into an operation contract	42
10.2	Implementation checks and retained evidence	42
10.3	Release, change and incident gates	44
10.4	Measurements that answer engineering questions	45
11	Synthesis and differentiation of Papers 1, 2 and 3	45
11.1	Research questions, methods and evidence	45

11.2	One workflow, three distinct obligations	47
11.3	Shared terminology and preserved distinctions	48
12	Deployment and implementation requirements	50
12.1	Placement of authority and supervision	50
12.2	Three feasible evidence modes	50
12.3	Conformance and implementation gates	51
13	Discussion and limits	51
A	Proposed record schema	53
B	Reproducibility and scope of the executable artifact	53
B.1	Inspecting a counterexample	54
B.2	Empirical reporting and reproduction limits	54
C	Executable reference gate and functional experiments	55
C.1	Artifact and implementation scope	55
C.2	Implemented contracts	55
C.3	Scenario design and observed results	56
C.4	Interpretation and limits	57
D	Provider-reported operational evidence	57
E	Extended proof obligations for a resource adapter	58
E.1	Evidence-view preservation	59
E.2	Recognition and semantic scope	59
E.3	Admission and physical object binding	59
E.4	Accounting, recovery and delayed closure	60
	References	61

1 Introduction

Two engineering organisations agree to run a simulation. The first controls a protected design file. The second operates the simulation service. Their agents can discover one another, authenticate and exchange a signed statement that the service passed an assessment. The first organisation still needs to answer several questions. Did the assessment cover the configuration that will receive this file? Is the assessor recognised for that property? Has an adverse finding appeared since the statement was issued? Does the agent have authority to release this particular file? What happens if the service accepts the job but its acknowledgement is lost?

These questions involve different objects. A signature authenticates an attributed statement. A policy decision evaluates that statement under a rule. An admission changes authoritative execution state. A remote receipt reports an event. An assurance claim concerns a bounded property under stated assumptions. A useful agent protocol must retain these distinctions when evidence crosses organisational boundaries and when some of it remains private.

This paper studies private verification of **action eligibility**. Eligibility requires both a current mandate and an adequately supported continuation for the same exact action. A mandate states what the represented principal permits. An assurance construction establishes what can be supported under the declared harm constraint. Neither creates the other. In a distributed workflow, the common object includes the initial operation and the continuation under which it can proceed, wait, fail or be contained. Two proofs about incompatible continuations do not establish a joint decision.

Paper 1 develops an epoch-bound deterministic controller for covered atomic digital commitments, including authority, exact-action, delegation, replay and shared-resource properties [1]. Paper 2 extends this account to partially observed multi-agent systems, pending effects and permanently charged continuation leases [2]. Its probability result assumes a coherent logical supervisor, an episode account, model containment and a restriction on the information used to select actions. The present paper makes the evidence boundary explicit. It asks how a relying domain can verify a decision without learning every supporting record, and how that historical decision remains applicable when it becomes an effect.

The central difficulty is selective evidence. A prover can reveal a genuine permission while omitting a genuine prohibition. It can also reuse a valid result after an assessment is withdrawn, substitute an unassessed deployment, or give two retries different message identifiers. Authenticity alone does not prevent these failures. The evidence view must be complete relative to declared sources and queries; its dependencies must include relevant absences; and admission must check current state at a protected boundary. The source registry can itself be wrong or incomplete. We therefore distinguish completeness of a committed query result from completeness of knowledge about the world.

A second difficulty is authority to assert. An organisation recognised to identify a legal entity is not thereby recognised to certify a deployed AI system. An accredited management-system certificate does not by itself establish a particular model’s safety. The protocol uses typed claims, scoped recognition paths and explicit derivation rules. It preserves the assessment object, method, deployment conditions, provenance and reliance limits. Each relying organisation decides which roots, methods and inference rules it accepts.

Our contribution is a connected specification with four parts. First, a bounded evidence semantics joins authenticated complete views, conservative mappings and claim-specific validation. Second, an application extension carries exact operation contracts, evidence challenges, proofs and typed execution records. Third, a protected state machine preserves current applicability, operation consumption, permanent debit and unresolved exposure. Fourth, analytical arguments and an

executable finite abstraction identify the assumptions under which those controls compose. These are composition results over established methods. We do not introduce a new proof system, a new general theory of trust, or the first consumable confidential mandate.

The manuscript distinguishes four forms of support. Propositions establish conditional statements about specified semantics. Exhaustive enumeration checks finite interleavings of selected controls. A separate executable reference gate tests concrete claim and admission functions on bounded fixtures. External examples show available evidence methods and assessment practices. The OpenClaw case is a documentation-based design study, not a deployed integration. None substitutes for deployment validation of a large language model, an independent certification of this protocol or a proof of the adequacy of its policy. The main case concerns two organisations, a bounded worker population, one named harm and one coherent joint assurance construction. Wider federation and decentralised risk composition require further arguments. Readers implementing agent systems can begin with the OpenClaw case and engineering guidance in Sections 8–10, then use Section 4 and the appendices to inspect the assumptions. Section 11 separates the contributions and research agenda of all three papers.

2 Related work and standards baseline

Cross-organisation execution joins several problems that existing research often studies separately. An agent must establish authority for an operation. A relying organisation must decide which external claims support that operation. Confidential evidence must remain meaningful when evaluated privately. Finally, the resources involved must preserve the decision’s conditions while authority, evidence and remote execution change. The present paper studies this last composition problem. Its construction reuses established cryptographic and authorisation mechanisms and introduces a bounded profile connecting them to claim validation, evidence dependencies and protected effect admission.

2.1 Authorisation proofs, confidential credentials and consumable authority

Proof-carrying authorisation provides an early foundation for separating proof construction from a small verifier. Bauer, Schneider and Felten describe distributed policies, challenge-based access, sessions and controlled disclosure of policy components [3]. Cyberlogic gives a logical account of evidential transactions, including attributed statements, delegation and revocation [4]. These approaches establish that distributed evidence and authority can be combined through explicit inference rules. Our use of a bounded verifier follows this tradition. Determinism makes a decision reproducible for fixed inputs; it does not establish that the inputs remain applicable at execution.

Private credential systems address a complementary boundary. ZKclaims supports confidential predicates over credentials, while zk-creds supports flexible statements involving existing identity infrastructure [5,6]. WAVE provides confidential, decentralised authorisation with transitive delegation [7]. These systems preclude treating private predicates or confidential authority chains as new contributions. Paper 3 instead asks which evidence view and issuer scope a private computation must preserve, and which later state transition may rely on its result.

Zero-knowledge authenticators also support policy-private authentication and, under an additional trusted-authority construction, oblivious policy updates [8]. This provides a further established comparator for confidential policy evaluation; it does not by itself establish current cross-domain evidence applicability or remote-effect closure.

Recent agent-specific proposals make this distinction more important. ERC-8354 defines confidential policy verdicts associated with exact actions and controlled consumption [9]. The

Zero-Knowledge Mandates specification develops consumable authority, including splitting, delegation and revocation. Its current history identifies revision 12 on 8 September 2026; separate non-financial effect profiles remain reserved [10]. Both are developing specifications. They require direct comparison rather than dismissal on maturity grounds. In particular, binding a proof to an action, protecting against replay, and conserving delegated authority are existing design directions.

The accounting object in this paper has a different purpose. A business allowance limits what an agent may spend or use. An assurance debit records the charge for issuing a complete conditional continuation, including pending effects and containment. Closing one operation does not replenish that risk account. This distinction is inherited from Paper 2 and supplies an additional composition obligation when several organisations enforce different local effects [2]. Paper 1 already separates a narrow decision kernel from a state controller; Paper 3 specifies how external claim validation and private evidence views enter that boundary [1].

2.2 Semantic evidence and claim validation

The difference between authenticity and completeness is established in authenticated-query research. Pang and colleagues study verification that returned records originate from the owner and that qualifying records have not been omitted [11]. This distinction applies directly to evidence-based admission. A valid signature on each disclosed approval leaves open whether a required prohibition or incident record was withheld. We therefore define completeness relative to named sources, bounded queries and accepted snapshots. It is a claim about a declared view, rather than completeness of all relevant facts in the world.

W3C standards provide useful, distinct components. Verifiable Credentials Data Model 2.0 represents attributed claims; Bitstring Status List provides a status mechanism [12, 13]. The PROV Ontology (PROV-O) describes provenance relationships, Shapes Constraint Language (SHACL) checks graph constraints, and Resource Description Framework (RDF) Dataset Canonicalization supports stable processing of graph representations [14–16]. A decentralised identifier can identify a subject and support key-related verification, while legal identity and mandate recognition require additional evidence and policy [17]. Our profile binds these objects to an assessment subject, deployed configuration, proposition, source scope and reliance decision.

The Boneh–Boyen–Shacham (BBS) cryptosuite offers selective disclosure and unlinkable derived proofs. The reviewed W3C text is a Candidate Recommendation Draft dated 10 September 2026 [18]. This supplies one privacy option for attributed evidence. The paper’s computation profile has a broader relation: the hidden evidence must satisfy the same accepted queries, claim rules and applicability conditions as the ordinary evaluator. Selective disclosure, private computation and a signed assessment assertion consequently remain separate evidence mechanisms. They may support the same admission policy through different trust assumptions.

Open Digital Rights Language (ODRL) 2.2 supplies policy concepts and a profile mechanism [19, 20]. It is a feasible interchange basis when the accepted fragment is explicit. The language also supplies conflict strategies: under *invalid*, a detected conflict voids the affected policy [19]. Our four decision outcomes, missing-evidence rules and aggregation procedure are defined by the proposed profile. They are not attributed to every ODRL implementation.

This restriction responds to active semantic work. The ODRL community’s formal-semantics document addresses evaluator behaviour. The Denotational Semantics and Axis-Aligned Semantics preprints examine knowledge-dependent and dimensional policy constraints [21–23]. Cedar separately demonstrates how a purpose-built authorisation language can support formal reasoning and an efficient implementation [24]. These are relevant comparators for the reference evaluator. The contribution sought here concerns preservation of an admitted meaning through

mappings, confidential computation and changing evidence. It does not depend on a claim to have introduced executable policy semantics.

2.3 Feasibility over current protocol foundations

The binding uses Agent2Agent (A2A) v1.0.0 as its reviewed documentation baseline. The official release list identifies v1.0.1 as a later release. The mutable current specification contains an additional in-task authorisation-scope clarification absent from the retrieved v1.0.0 page [25–27]. A concrete implementation must pin the release, schema and code revision independently of the website’s `latest` path.

A2A supplies structured message content, tasks and extension support. Its task identifier and message identifier serve communication purposes; the proposed operation and episode identifiers serve durable execution and accounting purposes. `AdmissionReceipt`, `EffectReceipt` and `ClosureRecord` are therefore application-profile records carried by A2A, not new names for A2A task states. In particular, optional `SendMessage` idempotency and terminal task behaviour cannot supply the resource-side guarantees required here [25].

The extension mechanism can host the profile without new core methods. However, the guide permits unsupported requested extensions to be ignored and recommends, rather than universally mandates, returning the activated extension set [28]. Covered execution therefore requires explicit agreement on the profile and refusal when that agreement is missing. This is a conformance requirement of the proposed extension.

Table 1: Existing foundations and profile obligations

Existing foundation	Useful role	Obligation of the proposed profile
A2A messages, tasks and extensions [25–28]	Exchange typed protocol content	Bind the exact effect contract; distinguish task status from effect status
Model Context Protocol (MCP) authorisation [29]	Protect HTTP tool access and apply resource audiences and scopes	Apply current mandate and assurance checks at covered tool effects
W3C verifiable credentials (VCs), status and BBS [12, 13, 18]	Represent attributed claims and selected disclosure	Specify accepted claim classes, issuer competence, freshness and deployment applicability
ODRL, SHACL and PROV-O [14, 15, 19, 20]	Express policies, validate shapes and describe provenance	Fix bounded evaluation, mappings, denial coverage and dependency meaning
OpenID Federation and Secure Production Identity Framework for Everyone (SPIFFE) federation [30, 31]	Establish federation relationships and workload trust	Constrain claim-specific recognition and preserve local resource authority

Continued on next page

Table 1 (continued)

Existing foundation	Useful role	Obligation of the proposed profile
Agentic Payment Protocol (AP2) mandate and receipt framework [32]	Represent scoped payment authorisation	Preserve the distinct joint continuation and unresolved-effect obligations
Assessment methods and competent bodies [33–40]	Produce evidence with defined technical or legal scope	Validate the assessment object, method, authority and permitted reliance

MCP is complementary and already supports operation-sensitive scope handling; its transport role should not be reduced to a binary identity check [29]. AP2 v0.2 uses Checkout and Payment Mandates and requires deterministic validation within its defined roles [32]. AGNTCY’s task-authority article distinguishes implemented standing permissions from task-bound profile work [41]. Together, these developments support an application profile while leaving its assurance semantics to be specified and evaluated.

2.4 Standards development and contribution boundary

The immediate requirement is a shared conformance target. Independent implementations need testable agreement on the effect manifest, evidence-query scope, semantic-profile version, claim category, assessor competence, validity conditions and failure outcomes. A vocabulary must preserve the difference between an attributed assertion, a test result, certification, cryptographic verification and regulatory determination. A common boolean field such as **compliant** would conceal those differences.

Further work should standardise the bindings between records, rather than require every ecosystem to use one issuer or ontology. Useful targets include snapshot and negative-query dependencies; deployment and configuration identifiers; persistent operation, episode and accounting domains; and separately typed admission, dispatch, effect and closure receipts. Recognition paths need explicit claim scope and local acceptance rules. Change notification must express which prior reliance is affected, including obligations retained after withdrawal. These requirements can be developed through interoperable profiles and test suites before changes to protocol cores are considered.

The paper’s results are accordingly bounded. They concern typed external claim reliance, complete declared evidence views, current exact-effect admission and persistent pending obligations under permanently charged continuations. Cryptographic soundness, source truth, calibrated harm models and correct resource adapters remain distinct assumptions or validation tasks. The following sections make those boundaries explicit in the system model and transition rules.

3 Evidence semantics and cross domain claim validation

3.1 Claims as bounded propositions

A claim is a proposition about an identified assessment object under conditions. Its representation carries the claim class, subject and configuration, predicate, context, method, supporting evidence,

issuer role, validity, status and reliance limits. The claim class determines the available inference rules. We distinguish operational alignment, safety, regulatory compliance, standards conformity and control effectiveness. Table 2 shows why these distinctions affect admission.

Table 2: Claim classes and their required scope

Claim class	Proposition to validate	Essential limitation
Operational alignment	Covered behaviour follows a principal’s agreed mandate and execution contract	Does not establish preference completeness or general model alignment
Safety	A named harm has a supported bound or is excluded within a specified model and environment	Depends on model, observation, timing and deployment assumptions
Regulatory compliance	An identified actor satisfies a governed set of applicable obligations	Requires role, jurisdiction, classification, assessment route and effective date
Standards conformity	An assessed object meets specified clauses under an assessment scheme	Scope and version of the standard and certificate remain binding
Control effectiveness	A named control meets a tested or analysed criterion	Effectiveness of one control does not establish overall system safety

A claim of operational alignment can be tested against an execution trace: every admitted release must follow the mandate and declared continuation. A claim of behavioural alignment, such as remaining within an intended task boundary during evaluation, needs a test definition and evidence. Neither warrants an unrestricted assertion that an AI system is aligned with human values. A system can follow a harmful policy faithfully. Policy adequacy and the interests of affected third parties remain separate questions.

Safety claims also need typed outcomes. A formal assertion that a bounded transition model excludes an event differs from an observed failure rate on a test dataset. A reported benchmark score differs from a calibrated deployment probability. The validation profile records the distinction and accepts only the inference that its evidence supports. A cryptographic proof can establish that a specified calculation was performed on committed inputs. It cannot make an inadequate model adequate.

For compliance, the evidence challenge identifies the applicable obligation set under a governed classification. It cannot let the prover select only convenient obligations. A provider, deployer, importer and distributor may have different duties. A record must bind the legal instrument and version, relevant role, system classification, assessment object, territorial and temporal scope, and assessment route. The European Union AI Act provides a concrete example of role-specific obligations and distinct assessment processes [42,43]. This protocol records and checks the accepted classification and assessment evidence. It is not a universal procedure for resolving contested legal interpretation.

The relying organisation also checks ongoing obligations. A pre-market assessment, where applicable, does not erase duties arising from a material change, monitoring result or incident. A system-level compliance claim therefore requires an explicit assurance case covering the applicable obligation set, organisational responsibilities, interfaces and change conditions. Conjunction of component certificates is not that argument. The NIST AI Risk Management Framework

provides governance context. ISO/IEC 42005 addresses AI system impact assessment; neither is a runtime permission token [44, 45].

3.2 Four valued validation

Let the validation result be Support, Reject, Indeterminate or Error. Support means that the specified relying policy accepts the bounded evidential inference. It does not mean that every underlying source statement is true. The profile checks seven families of conditions: authentication; issuer competence; applicability to the object and context; accepted method; adequate required evidence; current validity and status; and consistency with relevant adverse evidence.

These checks have explicit precedence. Malformed input, an unsupported operator or an invalid policy conflict produces Error. Otherwise an established blocking condition produces Reject. Otherwise an unresolved required condition produces Indeterminate. Support requires all mandatory checks to succeed. An applicable authoritative withdrawal is a blocking condition. An unresolvable syntactic or semantic conflict is an error. A missing report is unresolved evidence. Only Support can satisfy the corresponding claim prerequisite for an ordinary effect.

The operation evaluator has the parallel outcomes Permit, Deny, Indeterminate and Error. A required claim's Reject maps to Deny, and its unresolved or erroneous result cannot permit admission. The profile deliberately selects the ODRL invalid conflict strategy; its mapping to Error and the four-valued aggregation are this paper's semantics. The test corpus must fix the exact combination of multiple mandates, claims and prohibitions. Error and denial precedence does not create an execution difference, because both block ordinary admission, but it determines diagnosis and recovery.

Authentication distinguishes the original assessor from the party packaging evidence. A gateway may place an external report in a signed verifiable credential. That signature authenticates the gateway's representation and digest binding. It does not turn the gateway into the original assessor, nor establish that the report's original signature covers the extracted fields. The validation record identifies the source issuer, assessor, wrapper issuer, extraction method and signature coverage separately. A relying party can then require the original artifact, a trusted extraction process or an assessor-issued structured record.

Evidence categories are metadata, not a universal ranking. A claim can be self-asserted, backed by inspectable evidence, independently assessed, certified under a scheme, cryptographically verified, or the subject of a regulatory or judicial determination. Several categories can apply together. Independent testing and cryptographic verification answer different questions. Certification adds an institutional process with its own scope. A regulator's decision has authority under its legal remit. None automatically subsumes the others.

3.3 Semantic representation and the bounded policy profile

W3C verifiable credentials provide an attributed claim container [12]. The credential subject, holder, represented principal, agent instance, possession key and executor are distinct roles. The protocol binds the actual actor to the intended role and action. Decentralized Identifiers are optional identifiers, and control of a key is not evidence of organisational authority [17]. Credential status, mandate revocation, deployment invalidation and episode replacement are separate dependencies.

RDF represents the evidence graph. SHACL validates its structural constraints. PROV-O records derivation, activity and source relationships [14, 15]. These representations support interoperable records, but do not establish factual truth or source completeness. The `actedOnBehalfOf`

relation describes provenance responsibility; a validated mandate must separately authorise the action. Two reports derived from one sensor retain a common dependency even when different organisations sign them.

We define a proposed bounded ODRL 2.2 profile [19,20]. It reuses assigner, assignee, target, action, permissions, prohibitions, duties and selected constraints. Its executable subset contains finite membership tests, bounded integer or fixed-precision quantities, explicit units and time predicates with declared comparison semantics. The action taxonomy is versioned. Ambiguous units, unknown contexts and unapproved mappings fail closed. Input bounds cover parsing, graph construction and canonicalisation as well as rule evaluation.

Future duties are not treated as already satisfied preconditions. A recorded human approval can satisfy an admission condition. A promise to delete a file after processing creates an obligation with a deadline, observation method and authorised response. Its completion requires later evidence. The obligation remains in the continuation when its worker retires or a task ends. This distinction prevents a signed promise from being presented as proof of successful containment.

The proposed profile requires a bounded intermediate representation shared by ordinary and private evaluation. It excludes unrestricted ontology reasoning, unbounded recursion and fetching mutable semantic contexts during admission. Vocabulary definitions, canonicalisation, mappings and evaluator versions are pinned. RDF Dataset Canonicalization is an available foundation for graph identity [16]; using it still requires defined size limits, accepted document loaders and a binding from canonical records to the private relation. A production compiler must establish a conservative relation to the reference semantics. Finite tests can support that argument but do not prove it over all accepted inputs.

Table 3: Reused vocabulary and proposed extensions

Existing vocabulary	Role in the profile	Additional terms defined by this proposal
VC issuer and subject	Attribute statements and bind roles	Agent instance binding, assertion role, source and wrapper issuer
ODRL policy terms	Express bounded permission and constraints	Mandate root, delegation depth, exact operation and audience
PROV-O entities and derivation	Preserve lineage and shared dependencies	Source partition, query scope and evidence snapshot vector
Typed values and units	Canonical interpretation of quantities	Named harm, horizon, continuation debit and coverage envelope
Task and event identifiers	Correlate communication and observations	Durable episode, logical effect slot and consumption record
Assessment meta-data	Describe object, method and result	Claim class, competence scope, reliance limit and withdrawal reason

The additional terms belong to a proposed research namespace. They are not new W3C standard terms. A vocabulary registry can manage names, versions and test fixtures. It cannot prove that an accepted policy captures the intended harm or that an assessor’s methodology is sufficient.

3.4 Scoped recognition across trust domains

Each relying domain begins with an explicit local acceptance policy. Four relations remain separate: identity binding; mandate delegation; authority to make or recognise a class of evidence claim; and compatibility of assurance assumptions. A federation relationship in one relation creates no edge in another. OpenID Federation, workload federation and organisational identity mechanisms can supply useful authenticated paths [30, 31, 46]. The relying domain still decides what those paths permit it to infer.

A recognition path starts at an accepted root. Every intermediate issuer must have explicit authority to delegate recognition for the next edge. The effective claim class, object scope, jurisdiction, method and validity interval can only narrow. A finite depth bound and cycle rejection make the baseline decidable. A recogniser cannot acquire authority by routing its statement through a longer chain. The root decision and every edge remain auditable dependencies of the relying record.

For example, organisation A may recognise accreditation body R for a specified assessment scheme. R's record may establish that assessor C is competent within that scheme and scope. C's assessment of service B then requires a separate check of the assessed object, method, validity and applicability. Neither R's identity nor C's general status establishes that this particular deployment was assessed. B cannot issue an assessment in C's name by wrapping C's report. A also retains its own resource-release policy.

Mappings between domains must be conservative for the intended reliance. A unit conversion can be exact. An equivalence between two engineering-purpose categories may need a governed mapping with an explicit justification. Equality of a display label is insufficient. The formal model requires that permission under the mapped policy imply permission under the source policy for the translated action. Provenance and issuer scope are preserved independently. A mapping from a management-system claim to a system-safety claim requires substantive premises; a vocabulary alignment alone supplies none.

A supported cross-domain claim therefore has a finite derivation using three kinds of rule: an authenticated, recognised source assertion; an accepted conservative mapping; or an explicitly approved composition rule with its required premises. A system assurance case can include interaction assumptions and common-cause dependencies. Generic graph reachability, the number of signatures and conjunction of unrelated claims have no inference rule. The formal scope result below depends on these restrictions and on the adequacy of the accepted mappings and composition rules.

4 Formal model and conditional properties

4.1 Formal model and security boundary

The model connects a bounded evidence computation to an authoritative admission event and its subsequent effects. It refines the distinction between decision and state control developed in Paper 1 [1] and preserves the episode and continuation accounting of Paper 2 [2]. The analytical results concern the specified transition system. Their application to an implementation requires the refinement obligations stated below.

4.1.1 Controller lineage across the three papers

The deterministic controller concept continues Paper 1’s separation between a narrow decision kernel and protected state control, developed in its Sections 6 and 7.1 [1]. The kernel evaluates a declared relation over explicit inputs. The state controller maintains authoritative state and determines which transitions may occur. An optional prover and verifier establish a claim about the kernel’s computation; they do not perform the authoritative transition. We use *deterministic control architecture* for the connected evaluation, state-control and effect-enforcement functions, while retaining *state controller* for the protected state machine.

Paper 1 applies this separation to an atomic digital commitment. Its controller checks current authority, exact action binding, reservations and replay state before the covered commit. Fixed complete inputs, including authoritative state, produce the same specified control decision. Proposal generation, external revocation and request arrival can still vary. The invariant argument covers permitted proposal sequences and event orderings. The strategic result has separate game assumptions; controller determinism does not itself establish an equilibrium [1].

Paper 2 adds a runtime-assurance supervisor, with the controller connection specified in its Sections 9 and 14.7 [2]. It maintains model-indexed beliefs, evaluates candidate continuations and preserves one exposure-episode account through renewal and worker changes. An issued assurance lease binds a charged continuation that includes monitoring, containment and pending exposure. Protected execution enforces the mandate–assurance conjunction. Proposition 3 of Paper 2 states conditions under which added assurance guards and lifecycle transitions preserve Paper 1’s controller invariant. Deterministic inference and accounting can process probabilistic models without revealing the hidden state or establishing calibration.

Paper 3 specifies the cross-domain evidence and execution interfaces of this construction. Its pure eligibility evaluator applies a bounded semantic profile to explicit evidence and context. The protected state controller checks current applicability, consumes the operation slot and records pending work. The effect adapter mediates resource dispatch, containment and observations. Paper 2’s supervisor continues to supply the quantitative assurance construction; evidence verification supplies no replacement estimator. Local admission may precede a remote effect, so it is not Paper 1’s atomic resource commit. Preserving the earlier guarantees requires the control-refinement obligations in Section 4.6 and, for probability bounds, its additional stochastic-transfer conditions. The separation does not assume a deterministic environment, infallible evidence or instantaneous remote stopping.

4.1.2 Domains, operations and protected state

Let D be a finite set of independently governed domains. An exact local operation is

$$a = (p, i, k, d, b, r, o, h, q, u, e). \quad (1)$$

Here p is the represented principal, i the agent instance, k its authenticated possession-key binding, d the executor identity, b the recipient and audience binding, r the resource, o the operation, h the payload commitment, q the typed parameters, u the persistent logical operation slot, and e the exposure episode. The executor belongs to a governed domain $\text{dom}(d) = \delta \in D$; executor identity and domain identity remain distinct. The operation also commits to its contract, installed configuration, semantic profile and continuation position. A protected allocator binds u to an authorised business operation or single-use mandate slot. A caller cannot create a second entitlement by changing a message, task or instance identifier.

Mandate and assurance decisions must concern the same operation and qualifying continuation. Define their sets over a common universe of pairs (a, π) :

$$\begin{aligned}\mathcal{A}^{eligible}(S) &= \mathcal{A}^{mandate}(S) \cap \mathcal{A}^{assurance}(S), \\ \text{Eligible}(a, \pi, S) &\Leftrightarrow (a, \pi) \in \mathcal{A}^{eligible}(S).\end{aligned}\tag{2}$$

Agreement that an initial operation admits two different, incompatible continuations does not establish eligibility for their joint workflow.

The adversary may control planners, select favourable evidence, submit malformed policies, substitute contexts, replay messages and coordinate requests. The network may delay, duplicate, reorder or lose messages. A resource can continue previously admitted work while its acknowledgement is delayed. The baseline assumes authenticated accepted source records, protected authoritative state without rollback, and complete mediation of the declared effect family. Compromised source authorities, bypassed actuators and false observations remain explicit trust failures. Their consequences cannot be eliminated by proving a calculation over their outputs.

The complete state is

$$\begin{aligned}S &= ((S_\delta)_{\delta \in D}, E, Net), \\ S_\delta &= (\nu_\delta, U_\delta, Q_\delta, C_\delta, P_\delta, O_\delta).\end{aligned}\tag{3}$$

The vector ν_δ records relevant authority, evidence, policy, mapping and configuration versions. U_δ contains mandates and business-resource accounting; Q_δ contains claims, validations and status dependencies; C_δ records consumed operation slots; P_δ contains pending effects and response obligations; and O_δ is a durable outbox. E contains the coherent episode history or sufficient belief, active continuation, membership, pending-exposure index and append-only lease log. Its pending index is the exact projection of the participating P_δ records. Registering, changing or closing a pending obligation also advances the shared pending-state version within the same serialisation discipline. Net represents messages in transit. Retirement and replacement preserve episode identity, consumed slots, debits and pending obligations.

4.2 Bounded evidence and claim semantics

A semantic profile $\mathcal{P}$ fixes finite vocabularies, accepted sources, bounded queries, typed quantities, policy combination rules, claim derivation rules and resource limits. Its pure evaluator is $K_{\mathcal{P}}(a, \sigma, \theta)$, where σ is an evidence view and θ supplies every contextual input. There is no ambient network retrieval, mutable vocabulary context or hidden clock. Parsing, canonicalisation and arithmetic belong to the bounded computation.

For each required source partition s , the relying challenge fixes a root R_s , scope and version. In the baseline, the witness supplies the entire canonical finite record sequence committed by that root, together with its length and scope manifest. The reference view is

$$\text{View}_{\mathcal{P}}(a) = \bigcup_{s \in \text{Req}_{\mathcal{P}}(a)} \text{Query}_{\mathcal{P},s}(\text{Records}(R_s)).\tag{4}$$

The union retains source, partition, query and provenance tags; merging equal payloads cannot erase their attribution or create independent observations. The relation evaluates every required query over those sequences. Completeness is relative to the accepted sources and declared partitions. It does not establish that every relevant event in the world was reported. Membership

proofs for selected favourable records cannot replace complete-query validation. Authenticated range structures can later improve efficiency if their completeness property is established [11].

A claim has the form

$$c = (\tau, \varphi, z, \gamma, j, \rho, n, m, \eta, \iota, v, \Delta). \quad (5)$$

The fields identify claim class τ , proposition φ , assessed object z , configuration and use context γ , jurisdiction j , organisational or regulated role ρ , governing instrument and version n , assessment method m , evidence commitment η , source issuer ι , validity and status contract v , and dependencies Δ . A later evidence-wrapper issuer remains distinct from the original assessor.

For relying domain δ , validation returns

$$\text{Val}_{\delta,S}(c) \in \{\text{Support}, \text{Reject}, \text{Indeterminate}, \text{Error}\}. \quad (6)$$

Support requires source authentication, recognised competence, deployment applicability, an accepted method, the required evidence, current status and consistency. Malformed or unsupported input produces Error. Otherwise an established blocking condition produces Reject; unresolved required information produces Indeterminate; and satisfaction of every requirement produces Support. The profile specifies how contradictions are classified. Only supported claims required by the action profile can contribute to Permit.

Support means that the claim meets the stated reliance rule under its inputs. For an external assessment, the directly authenticated proposition may be that a named body issued a specified finding. Inferring deployment safety additionally requires an adequate method and applicability argument. For compliance, a governed function fixes the complete applicable obligation set:

$$\mathcal{O}_{app} = \text{Obligations}(n, j, \rho, z, \gamma, t). \quad (7)$$

Its classification inputs, authority, assessment route and effective time are part of the claim profile. A prover cannot choose a convenient subset of obligations. The protocol retains the record's stated legal scope, issuing authority and reliance limits; legal effect remains governed by applicable law.

Claims are derived through three rules. A **Source** rule accepts an authenticated assertion only within its issuer's recognised class and scope. A **Map** rule permits a versioned transformation only when it preserves source provenance and satisfies a proposition-specific conservative implication. A **Compose** rule derives a system claim only through an approved assurance-case rule whose premises, interactions and common dependencies are checked. Signatures and graph reachability supply no additional inference rule.

The claim dependency set Δ includes positive premises and absence-sensitive conditions. A validation that relies on the absence of a reported incident must identify the accepted incident source, query scope and status version. A later incident, deployment change or revised classification can invalidate future reliance without changing the authenticity of the original assessment. Historical validation records retain their original inputs and result. Their lifecycle is therefore distinct from the source claim's lifecycle and from the execution state of an operation already admitted on that basis.

For the baseline, any update to a declared relevant partition invalidates dependent validation. More selective reuse is justified only if every query result read by the evaluator remains unchanged, including empty ranges, type membership and denial categories, and if no external admission guard changes. Under those conditions deterministic reevaluation has identical inputs and

therefore the same result. Tracking only records present in a successful witness does not establish the condition. This conservative partition rule supplies a reference against which a future dependency extractor can be tested.

4.3 Private evaluation and scope preservation

Public input x binds the exact operation and continuation, semantic and evaluator versions, required roots and queries, trust-policy version, episode context, assurance and lease mode, audience, actor binding, challenge and applicability interval J . Its witness w contains private credentials, source sequences, accepted mappings and the evaluation trace. The proposed relation is

$$R(x, w) = \text{Src} \wedge \text{View} \wedge \text{Sem} \wedge \text{Mand} \wedge \text{Assur} \wedge \text{Bind}. \quad (8)$$

The predicates establish accepted sources, the complete required view, bounded semantics, mandate satisfaction, the selected assurance check and exact context binding. The evaluation predicates include the interval condition

$$\forall t \in J : \quad K_{\mathcal{P}}(a, \sigma, \theta[t]) = \text{Permit}, \quad (9)$$

where $\theta[t]$ replaces the declared time input by t while retaining all other committed inputs, including continuation π . In issued-lease mode, θ contains the canonical context authenticated by the existing lease receipt. In prospective mode, x commits to a complete predecessor E_- , a proposed issue-or-renew transition and its canonical successor E_+ . The relation checks that the specified guarded transition maps E_- to E_+ , and evaluates (9) using the successor context θ_+ . It proves a permissible hypothetical transition, not its occurrence. Admission must authenticate the actual receipt and match every evaluation-relevant successor field. Receipt signature bytes are verified separately from those canonical fields.

The baseline accepts only a supported time fragment for which (9) reduces to bounded comparisons or an accepted conservative interval calculation. Admission checks $[\hat{t} - \epsilon_{clk}, \hat{t} + \epsilon_{clk}] \subseteq J$, where $\hat{t}$ is its clock reading and ϵ_{clk} the declared clock-error bound. Thus a proof about an earlier instant alone cannot support later admission merely because database versions remained unchanged.

The computation profile proves execution of the specified bounded assurance calculation. The assertion profile validates an accepted assurance service's exact-context assertion; that service remains trusted for its calculation. Neither profile establishes model adequacy from cryptography alone. Relevant private information used to select an action must enter the declared assurance information model, preserving Paper 2's restrictions on proposal selection [2].

Proposition 1 — Relative view and decision fidelity. Suppose commitments bind the canonical records and context; the relation checks all required finite source sequences, queries, accepted issuers and bindings; and the proof system is computationally sound. Suppose also that the relation executes $K_{\mathcal{P}}$ exactly, or uses a compiler with the separately established property

$$\widehat{K}_{\mathcal{P}}(a, \sigma, \theta) = \text{Permit} \Rightarrow K_{\mathcal{P}}(a, \sigma, \theta) = \text{Permit}. \quad (10)$$

Except on the authentication, binding or proof-soundness failure events, acceptance of a proof of Permit implies reference Permit on the uniquely committed required view and context. An applicable denial cannot be removed merely by omitting it from the witness.

Proof. Soundness excludes acceptance of an input outside the relation language. Binding fixes every canonical source sequence and contextual commitment. The required-source loop and complete query evaluation therefore reconstruct the reference view, including absence-sensitive queries. Exact evaluation, or (10), gives reference Permit. Omitting a committed denial either changes the authenticated sequence, falsifies the query computation, or leaves the denial in the reference view. None yields a valid permitting witness. $\square$

This result establishes existence of a satisfying evaluation witness. Proof of witness knowledge requires a stronger primitive and argument. In the assertion profile, its conclusion concerns faithful validation of the signed assertion, while the underlying calculation remains an external premise. Cryptographic failure probabilities are understood relative to a stated security parameter, adversary class and number of verification attempts. Applying the result across polynomially many adaptive attempts requires the corresponding security notion and failure bound; the later control arguments condition on the absence of those failures.

Recognition is claim-specific. A root grants capabilities of the form $(\iota, \tau, \Omega, n_{rec})$, identifying issuer, claim class, scope and remaining recognition-delegation depth. A signed extension is valid only when its parent has recognition-delegation authority, the depth decreases, and the child scope is intersected with inherited and relying-domain scope. Approved conservative mappings interpret every scope in a common representation. For source policy P , mapping m and accepted target action a , the directional condition is

$$\text{Permit}_{mapped}(m(P), a) \Rightarrow \text{Permit}_{source}(P, \overline{m}(a)). \quad (11)$$

The action translation $\overline{m}$ must be defined on accepted inputs and preserve the relevant identity, resource, quantities and provenance. A corresponding claim transformation requires an implication for its specific proposition and assessment context. The baseline rejects cycles and bounds path length. For an accepted path,

$$\Omega_{eff} = \Omega_{root} \cap \Omega_{\delta} \cap \bigcap_k \Omega_k. \quad (12)$$

Proposition 2 — Rooted scope preservation and claim non-escalation. Every capability derived by these recognition rules remains within its root’s claim classes and effective scope. Under the Source, Map and Compose rules, an identity assertion or management-system certificate alone cannot derive an unrelated system-safety claim or regulatory determination.

Proof. Root capabilities satisfy the first statement. Each extension selects an authorised parent class and intersects scopes; therefore its result is contained in its parent. Induction through the finite path proves containment in the root and relying scopes. Conservative mappings preserve that containment and the authenticated source. For the second statement, inspect the final claim-derivation rule. Source preserves the assertion’s class, proposition and assessed scope. Map requires the named conservative implication. Compose requires its distinct substantive premises and recognised authority. No rule derives the unrelated proposition from a signature alone. $\square$

The adequacy of an accepted semantic implication or assurance-case rule remains a substantive premise. Type checking prevents unlicensed inference; it does not prove the correctness of every authorised engineering or legal inference.

4.4 Authoritative admission and continuation state

The deterministic control architecture has three logical functions: pure eligibility evaluation, protected admission and effect enforcement. This preserves the decision-kernel and state-controller

distinction established in Section 4.1.1. A proof establishes an evaluation or proposed transition, subject to verification of its agreed relation. The protected state controller must accept that transition, and the resource-specific effect adapter must bind it to dispatch and observed outcomes. Paper 2’s supervisor supplies the joint belief, qualifying continuation and coherent episode account on which these local functions rely [2]. The functions produce different records and need not reside in one process or organisation.

Let lease ℓ identify continuation π_ℓ , debit b_ℓ and coverage envelope $\mathcal{K}_\ell$. The envelope names admissible operations, retained pending obligations and permitted transition conditions. Its structural coverage is distinct from a numerical safety argument. The state machine uses the following transitions.

Table 4: Abstract control transitions

Transition	Required guard	State change
Issue or re-new	Fresh lease; authorised episode service; sufficient allowance; matched predecessor; pending and transition exposure included	Append debit and lease; advance episode state; install through a barrier or covered overlap
Admit	Valid result; applicable versions and time; current mandate and required claims; fresh operation slot; active coverage; available business resources	Atomically consume slot, reserve resources, record admission and register pending outbox effect
Dispatch	Existing admitted outbox record; unchanged effect binding; applicable protected fence or explicitly covered dispatch obligation	Submit the same logical operation and record dispatch
Observe	Authentic, correctly typed resource event for the operation	Update observations; retain pending exposure until closure conditions hold
Close	Resource-specific evidence satisfies the declared closure predicate	Close pending work and release eligible business reservations; retain consumption and lease debits
Contain	Scoped protective authority and covered response	Block new ordinary work and invoke protection while retaining unresolved effects
Update or replace	Governed status, version or lifecycle event	Invalidate affected future admissions; preserve episode history, consumption, debits and pending work

These are transitions of one coherent logical ordering. Where several services implement it, an explicit serialisation protocol must provide the same property. An A2A exchange or signed hypothetical account transition does not supply atomicity.

Define $\text{Current}_\delta(x, S)$ by comparison of all relevant authoritative versions, accepted status age, actor and deployment bindings, applicability interval and required peer obligations. Negative-query partitions belong to the compared versions. A denial inserted into a formerly empty required category therefore invalidates an old decision. Current is relative to the specified

publication, delivery, polling and expiry contract; it does not mean instantaneous knowledge of remote changes.

A prospective proof evaluates the proposed successor context as defined above. The protected service's receipt must authenticate that exact transition from its committed predecessor. Admission compares actual state to this certified successor and compares every unaffected coordinate normally. It never treats a permitting evaluation of the predecessor as a permitting evaluation of an arbitrary successor. Any change to pending work, continuation, debit, configuration or another relevant field requires the same exact match or revalidation. A proof of an already issued lease instead binds its existing receipt.

Proposition 3 — Applicable and unique local admission. Outside the failure events of Proposition 1, and under the declared freshness and state-ordering contracts, each successful admission has a permitting evaluation for its exact operation and the authoritative relevant state at its linearisation point, together with an issued covering continuation. Each protected operation slot is admitted at most once.

Proof. Admit compares the proof bindings, time window, relevant versions and state guards in the same serialised transition that consumes the operation slot. The actual admission time lies in the certified interval under the clock assumption, so (9) supplies the time-dependent permitting evaluation. An earlier relevant change causes a comparison or validity failure. A later change does not alter the historical admission. In the prospective branch, the proof evaluated θ_+ and the authenticated receipt and current-state checks establish that these are the actual successor fields. Thus this branch also has a permitting evaluation of the admission context. For uniqueness, order two alleged admissions for the same slot by their linearisation points. The first inserts the slot into durable C_δ . No transition removes it; hence the second fails its freshness guard. $\square$

Peer readiness is an authenticated bounded obligation under the withdrawal and delay contract. It is not a globally atomic observation. Similarly, local admission and remote effect commitment have separate times. Requiring a fresh dispatch decision is an application choice that must be represented in the continuation and adapter guards.

4.5 Accounting, pending effects and operational conformance

For the append-only issued-lease log L , remaining allowance is

$$B(S) = B_0 - \sum_{\ell \in L(S)} b_\ell, \quad b_\ell \geq 0. \quad (13)$$

Every new or renewed lease charges its full supported continuation. Rechecking an existing lease adds no debit. Business-resource accounting remains separate. For mandate root μ ,

$$\text{spent}(\mu) + \sum_{u \in \text{reserved}(\mu)} \text{amount}(u) \leq \text{limit}(\mu). \quad (14)$$

Business accounting follows the resource contract. While a remote charge is unresolved, the corresponding reservation remains held. Authenticated completion transfers the bound charge from reserved to spent; established non-commitment may release the unused reservation. A cancellation request alone establishes neither outcome. This preserves the resource meaning of (14) across the non-atomic boundary introduced after Paper 1. None of these transitions refunds an issued assurance debit. Preparation that itself creates the named exposure must already be covered.

Proposition 4 — Permanent debit and pending retention. From a valid initial state, every transition trace satisfies $0 \leq B(S) \leq B_0$. Issued debits persist. A consumed effect with unestablished closure retains its pending record. Every activated replacement continuation structurally includes existing pending obligations and installation exposure.

Proof. Initially the properties hold. Issue and Renew are the only transitions that change L ; each appends a nonnegative debit bounded by the remaining allowance. Thus B decreases and stays nonnegative. Admit registers pending work before dispatch is possible. Observation, cancellation, replacement and renewal retain that work; only a satisfied closure predicate permits closure. A replacement continuation checks the current pending set and transition conditions before installation. Any concurrent addition changes the checked state and invalidates a stale installation. Induction over the transition sequence proves the claims. $\square$

The proposition preserves accounting and structural obligations. It does not prove physical containment or calculate their probabilities. A terminal task signal is insufficient for closure unless the resource-specific predicate makes it sufficient. A residual-tail policy requires retained tail state and a separately justified bound.

At-most-once remote commitment additionally requires the receiver to atomically check and persist the same operation slot at the effect boundary. The serialisation argument from Proposition 3 then applies there. A sender outbox alone cannot establish this property. Completion under permanent failure is outside the baseline guarantee.

Operational alignment is defined as bounded trace conformance. Let $F_I(\xi)$ be the covered effects in interval I of trace ξ , and let $\text{Adm}_\xi(f)$ contain the admission events that bind effect f to its persistent operation slot. An event ω records operation a_ω , continuation π_ω and state S_ω^- immediately before its authoritative transition. Write $V(f, \omega)$ for the conjunction of $\text{Eligible}(a_\omega, \pi_\omega, S_\omega^-)$, $\text{Matches}(f, a_\omega)$ and $\text{Follows}(f, \pi_\omega, \text{updates}_f)$. Define

$$\text{Align}_I(\xi) \Leftrightarrow \forall f \in F_I(\xi), \exists \omega \in \text{Adm}_\xi(f) : V(f, \omega). \quad (15)$$

Eligibility is evaluated immediately before admission consumes the slot. An effect without a corresponding admission fails the predicate. Follows covers subsequent dispatch, authorised generation changes and protective continuation. This permits already admitted remote work to remain pending after revocation while new ordinary admissions stop. Establishing (15) requires complete mediation and authentic, complete observation of the declared effect family. It establishes neither hidden goal alignment nor behaviour outside that family.

4.6 Refinement, stochastic transfer and privacy

Let $\mathcal{A}$ be the specified abstract machine and $\mathcal{C}$ a concrete implementation. A relation $\mathcal{R}$ and projection α must relate concrete durable state and actual pending work to abstract state. The proof obligations are: related valid initial states; one matching permitted abstract transition or genuine state stutter for every concrete step; exact reflection of every covered external effect; and preservation of these conditions through recovery, duplication, rejection and replacement.

Proposition 5 — Conditional safety-trace refinement. If these obligations hold for every reachable concrete state, each finite concrete execution projects, after permitted stutters, to a trace of $\mathcal{A}$:

$$\alpha(\text{Traces}_{fin}(\mathcal{C})) \subseteq \text{Traces}_{fin}(\mathcal{A}). \quad (16)$$

Every abstract invariant expressed over α then holds in the implementation.

Proof. The initial-state obligation supplies a related abstract start. Assume a concrete prefix has a matching abstract trace. Its next step either preserves the projected state as a stutter or extends that trace by the permitted matching transition. Effect reflection prevents removal of an externally relevant event. Induction establishes trace inclusion and the invariants. $\square$

This is a conditional forward-simulation result. The substantive obligations concern evidence-view fidelity, negative-dependency invalidation, scoped recognition, admission atomicity and pending-state preservation. A proof for an actual implementation remains required. The finite checker evaluates a restricted abstraction and does not discharge these implementation obligations.

Safety-language inclusion alone cannot transfer a stochastic harm bound. Identical permitted traces can have different probabilities. Paper 2 distinguishes a continuation’s conditional incremental first-harm bound from the probability bound for the whole episode [2]. For a candidate lease ℓ issued at time t with continuation π , first-harm time τ_H and terminal exposure time T_{close} , the local claim under model coverage is

$$\Pr_{\theta^*}^{\pi}(t < \tau_H \leq T_{close} \mid \mathcal{I}_t) \leq r_t(\ell) \leq b_{\ell}. \quad (17)$$

Here $\mathcal{I}_t$ is the declared history, including the fixed calibration record, and the probability law is that of the true retained model θ^* under the candidate continuation π . The quantity $r_t(\ell)$ is Paper 2’s maximum continuation value over its retained model bank; b_{ℓ} is the permanent debit and must cover that value. The posterior retains mass for previously unobserved harm. The event in (17) excludes earlier harm by definition; it does not condition on unobserved survival. Proposal selection and renewal use the declared information filtration. Additional private selection information requires an expanded observation model, its selection likelihood, or the conditional-independence condition of Paper 2.

Transfer requires a coupling or conservative dominance of transition, observation, timing and harm laws that preserves this information and renewal structure. Proof generation and network waiting can add exposure despite being logical state stutters. With initially absent harm, a containing model bank, independent calibration and permanent pathwise debits whose sum is at most ε , Paper 2 bounds first harm over the complete episode by $\varepsilon + \delta_{cal}$, where δ_{cal} is its calibration coverage error δ [2]. An initial harm allowance must be reserved or added as specified there. This episode bound is taken over deployment and calibration randomness; it is not an ε bound conditional on every later history, nor is remaining allowance automatically a posterior risk bound. A separately justified implementation-failure bound may be added by a union bound. Unmeasured defects receive no invented probability.

Privacy is a separate property of the selected proof construction and disclosure policy. Computational zero knowledge requires an efficient simulator, without the private witness, whose output is computationally indistinguishable from the verifier’s real view under the chosen setup, auxiliary-input and composition model. The simulator receives the public statement and any inputs explicitly allowed by that model. Public roots, operation identifiers, assessment categories, outcomes, sizes and timing may still reveal information. Repeated challenges can reveal additional classifications even when each proof meets its individual definition. The protocol must therefore declare its leakage function, query authorisation, challenge limits and error disclosure. The present construction specifies the relation and these requirements; it reports neither an implemented proof backend nor a complete protocol-level privacy proof.

5 Protocol and protected execution

5.1 Negotiation and operation binding

The protocol is carried by a required, versioned A2A extension. Its records are proposed application objects, not new core A2A remote procedure calls. A receiving gateway rejects covered work unless the parties explicitly acknowledge a compatible extension profile. The acknowledgement binds the schema, semantic profile, proof verifier, assurance mode, record formats and failure behaviour. Extension activation is conveyed through the transport binding's service parameter; a URI in Message data alone is insufficient. This stricter negotiation rule prevents silent downgrade [25, 28].

Discovery identifies a service and its declared capabilities. A signed Agent Card can support discovery authenticity. It does not grant the discovered service authority to assess arbitrary claims or authorise access to a relying organisation's resources. Each party authenticates the peer, represented organisation and negotiation transcript under its local policy. Private evidence is disclosed only to an authorised audience and for the agreed purpose.

The OperationContract identifies the exact effects on both sides. It fixes persistent logical operation and episode identifiers, principals, agent instances, possession keys, executors, recipients, resources, payload digests, typed quantities, policy and configuration versions. It also identifies the common qualifying continuation, time limits, observations, failure branches and containment responsibilities. The business agreement, mandate, assurance construction and protocol contract remain distinct objects even when they refer to one another. Server-assigned task identifiers can differ between parties and are linked when available.

The EvidenceChallenge fixes what must be established before reliance. It names required source authorities and partitions, query definitions, roots or acceptable root-selection rules, required denial coverage, accepted claim profiles, freshness bounds, proof mode, challenge and audience. For safety or compliance it also identifies the assessment object, deployment context, accepted methods, competence scopes and governed obligation set. The prover cannot narrow these requirements by presenting only favourable evidence. Evidence disclosure or service provisioning that can itself cause the named harm must already be covered by an appropriate continuation.

5.2 Evaluation and readiness

A prover prepares the committed evidence view and runs the selected evaluator. The private-computation mode proves the specified relation. The assertion mode validates an accepted service's signed exact-context result. These modes have different trust premises and are explicitly negotiated. Selective disclosure is a third mechanism for revealing only needed credential statements; it does not imply the richer private-computation relation. A verifier records the mode, accepted roots, evaluator identity, output and dependencies.

The EligibilityProof establishes a historical computation claim. It does not consume authority, reserve a business resource or establish that a live lease has been charged. The proof indicates whether it refers to an already issued lease or a prospective issuance. In the issued-lease branch, generation occurs after the required issuance. In the prospective branch, it commits to the exact predecessor and proposed issuance transition; an authenticated receipt identifies the permitted successor. All unrelated changes still require revalidation. This distinction prevents both circular charging claims and a general exemption for stale proofs.

Readiness is a protected obligation, not an informal promise by a language model. The receiving resource issues a ReadinessRecord naming the operation, continuation, accepted receiving

conditions, expiry and withdrawal procedure. It specifies what happens to reliance already accepted before withdrawal. The record does not make A and B one atomic database. The joint continuation covers partial completion, refusal, delay, lost acknowledgements and a failure between the two local admissions.

The episode service issues or renews the covering lease before exposure. Every new lease permanently charges its complete continuation, including setup, installation, pending effects and containment. Checking an existing covering lease at another gateway creates no debit by itself. A change that requires a new continuation requires a new debit. Installation uses the specified barrier or a justified overlap policy. Business reservations may later be released; the accumulated risk debit is retained.

5.3 Admission and dispatch

Figure 1 makes the three functions of the deterministic control architecture explicit in each domain. The proposing agent suggests an action and prepares evidence outside the protected execution-control boundary. Function 1, pure eligibility evaluation, computes the declared decision over explicit inputs; an optional proof is checked by a distinct verifier before reliance. Function 2, protected admission, belongs to the state controller. It checks current applicability and determines whether the operation slot can be consumed. Function 3, effect enforcement, belongs to the resource-specific effect adapter. It mediates the covered resource boundary under the resource owner’s authority. These responsibilities and their failure assumptions remain separate even when components are co-located.

The shared logical assurance service carries Paper 2’s joint continuation and episode semantics. It exchanges lease, debit and pending-state information with protected admission; it does not bypass local mandates to operate a resource. Typed resource observations return through the adapter to local and joint state. Acceptance, completion, cancellation and verified closure have different meanings. Missing or insufficient evidence leaves the effect pending, and closure does not refund the permanent risk debit. The arrows specify logical dependencies, not a globally atomic network transaction.

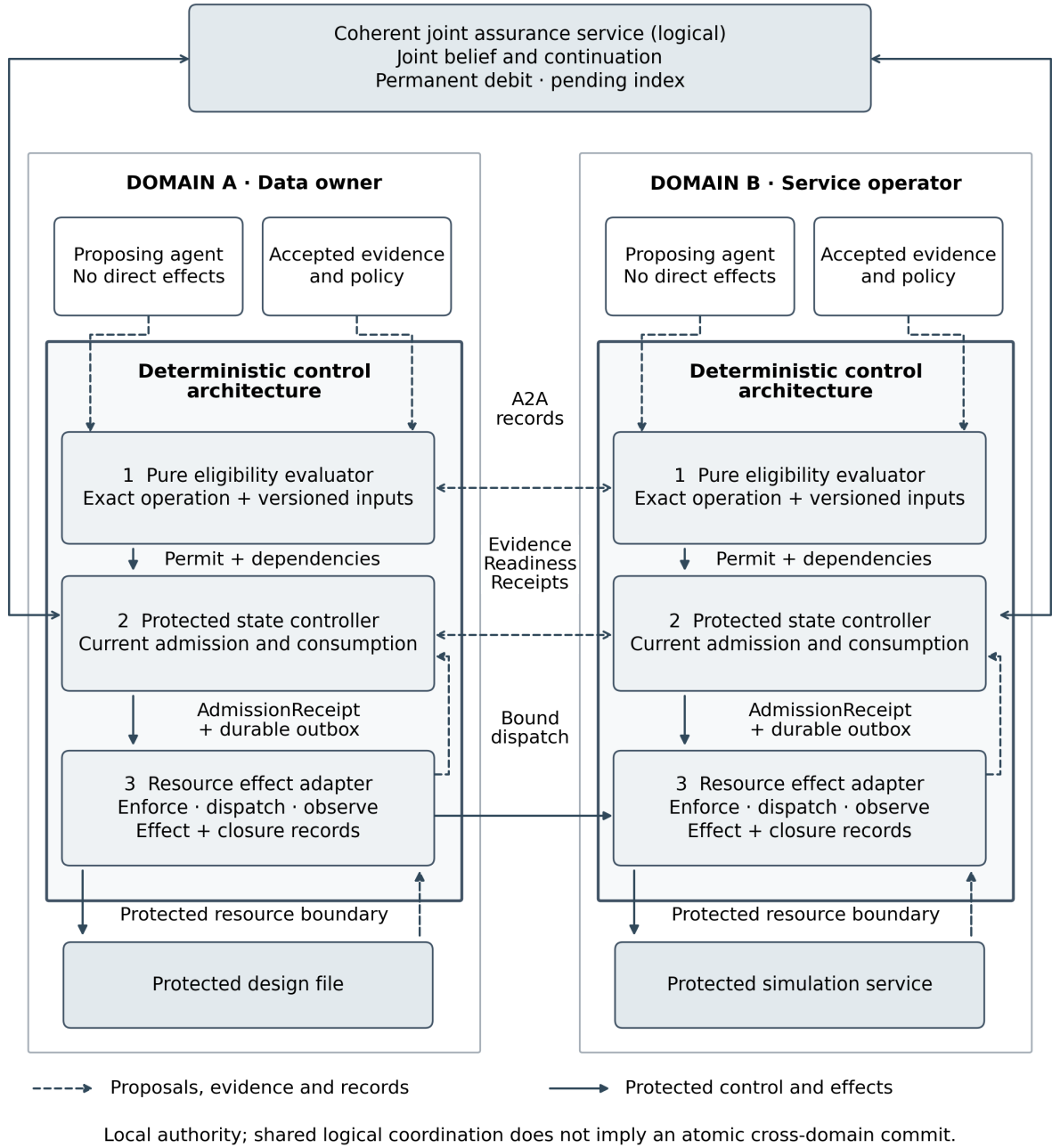


Figure 1: Deterministic control architecture across two trust domains. Each domain separates pure eligibility evaluation, protected admission by its state controller, and effect enforcement by its resource adapter. The shared logical assurance service preserves the joint continuation and episode account; local resource authority remains separate. Dashed paths carry proposals, evidence and records, including resource observations. Solid paths show protected control and effects. Distributed serialisation and effect binding remain explicit implementation obligations.

An AdmissionRequest carries the exact contract and proof bindings. At the serialisation point, the controller checks current authority, status and relevant versions; the proof’s applicability interval; the active continuation; required peer readiness; available business resources; and the unconsumed logical operation slot. A successful transition consumes that slot and durably records the admission, reservation, pending obligation and outbox item. These updates belong to one protected transaction or an implementation with an equivalent serialisation argument.

The outbox gives recovery a durable instruction. It does not prove that the receiver committed the effect. The adapter dispatches only the bound operation, using the same resource-level idempotency key on every retry. It checks the current dispatch fence or follows a previously committed dispatch obligation explicitly retained by the continuation. The receiver independently applies its authority and effect-boundary rules. At-most-once commitment requires protected receiver-side consumption; sender-side duplicate suppression alone cannot establish it.

An admission receipt means that the local authoritative state accepted an operation. A dispatch record means that an adapter submitted it. An effect receipt identifies a particular resource event or observation. These types cannot be substituted. A gateway that authenticates a proof is not thereby authorised to issue a resource’s effect receipt. A resource receipt should distinguish acceptance, execution, completion, cancellation and closure evidence.

5.4 Revocation and the containment protocol

Revocation blocks new ordinary admissions under the revoked authority. It can also block dispatches that remain locally controllable. It cannot retroactively undo a remote commitment or prove that a queued action will never execute. Figure 2 shows why the admission time and effect time are different points in the argument. A stale proof is refused for new work, while earlier admitted work remains an outstanding obligation.

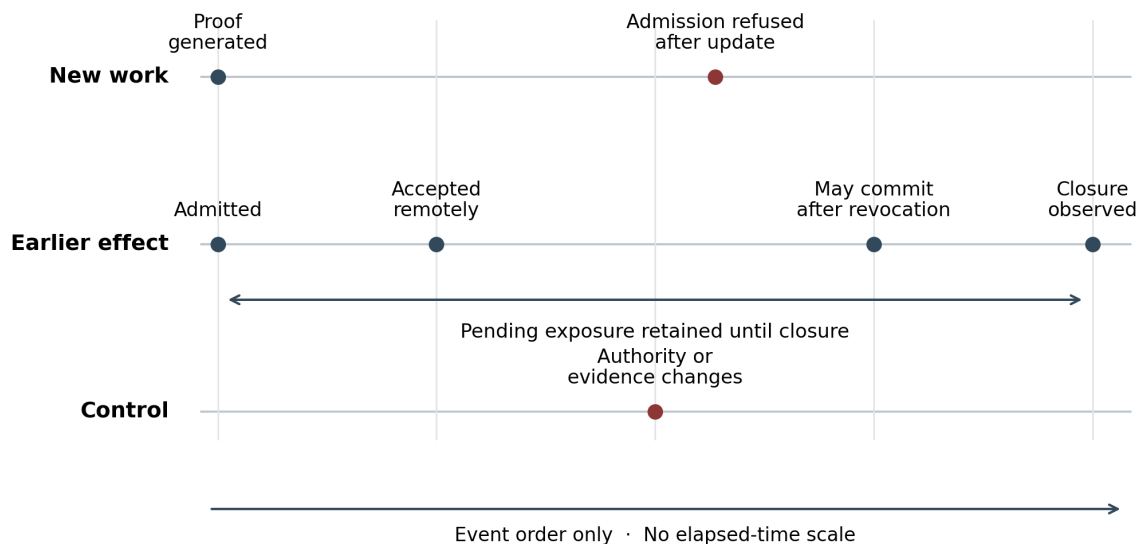


Figure 2: A relevant adverse update invalidates future admission while an earlier remote effect remains pending. Revocation changes ordinary authority; verified closure requires a resource-specific observation.

The kill switch is therefore a containment protocol. A scoped ContainmentRequest authenticates the initiator, target episode, affected resources, reason, requested response and escalation deadline. The protected controller advances the ordinary-work fence, stops new proposals from becoming

effects and invokes the already charged protective continuation. Protective authority is defined independently, so revoking ordinary release permission does not unintentionally revoke the authority to quarantine a service or revoke its keys.

The resulting status is explicit: requested, locally fenced, remotely acknowledged, containment in progress, verified closed, or unresolved and escalated. Acknowledging a cancellation request does not imply successful cancellation. An inability to observe a remote resource remains uncertainty in the pending record. Restart requires a governed decision, renewed applicable evidence and a covering continuation; it does not reset the episode debit or remove consumed operation identifiers.

ClosureRecord establishes the declared resource-specific closure predicate. For the engineering case, it may require confirmed non-dispatch, acknowledged cancellation before release, or completion and verified disposition of retained data under the contract. An application may permit closure with a bounded residual tail only if the tail remains represented and its bound is separately justified. The finite artifact in this paper models terminal closure, not this broader residual-tail construction.

A2A task completion and cancellation remain communication states. The released protocol rejects new messages on terminal tasks; subsequent reconciliation uses a linked control task or a protected status interface [25]. Task identifiers, worker identities and proof identifiers may change. The original logical effect and episode identifiers persist. Retirement transfers outstanding obligations to a supervised successor or reconciliation service.

5.5 Record types and failure reporting

Table 5: Proposed protocol records and authoritative meaning

Record	Meaning	Required authority
OperationContract	Agreed exact effects and continuation	Parties for their own obligations
EvidenceChallenge	Required evidence and reliance profile	Relying policy authority
ClaimProfile	Proposition, method and acceptance semantics	Accepted profile governance
EligibilityProof	Stated computation over bound inputs	Accepted proof relation and verifier
ClaimValidationRecord	Bounded reliance decision with dependencies	Relying verifier under local policy
ReadinessRecord	Resource accepts a bounded preparation obligation	Governing resource authority
LeaseRecord	Continuation issued and debit recorded	Coherent episode accounting authority
AdmissionReceipt	Local protected state consumed an operation	Local admission controller
DispatchRecord	Exact operation submitted	Protected effect adapter

Continued on next page

Table 5 (continued)

Record	Meaning	Required authority
EffectReceipt	Typed resource event or observation	Accepted resource or observer
ClosureRecord	Declared exposure predicate established	Accepted closure authority and evidence

Every authenticated record has a type and version, issuer role, subject, context binding, validity or event time, and status or replacement mechanism. The application defines typed reasons such as unsupported profile, invalid evidence, unresolved evidence, stale context, withdrawn authority, consumed operation and pending outcome. These are extension semantics; their mapping to transport errors is part of the profile. Recovery always reconciles the original operation before requesting another effect. Rate limits, bounded parsing and authenticated challenge issuance reduce denial-of-service and policy-probing exposure, but do not establish availability against an unbounded adversary.

6 Executable validation

6.1 Model and method

The companion Python program performs breadth-first exploration of a finite transition system. A state records authority and denial status, lease and accumulated debit, relevant sequence versions, two proof-request slots, consumption, pending exposure, business reservation, remote progress, observations, cancellation, lifecycle and diagnostic invariant flags. Each enabled event yields a successor state. Already visited states are deduplicated. The baseline exploration continues until no new state remains; each ablation stops at its first invariant violation and returns a shortest event trace.

The bounds are two request identities for one logical operation, one initial lease and at most one renewal, two abstract allowance units, one authority revocation, one adverse-evidence insertion, and one retirement followed by replacement. The remote operation can accept, commit, complete or confirm cancellation independently of delayed observations. Duplicate delivery is represented separately. Lease units stand for externally justified charges; they are not estimated probabilities.

Proof construction is abstracted by recording a permitting snapshot. The program does not execute a cryptographic prover, graph parser or claim validator. Pending coverage is represented by an integrity flag that detects an intentionally omitted obligation during renewal. It does not enumerate a set of physical obligations or calculate the continuation’s safety. These abstractions isolate the state controls whose interleavings are checked.

6.2 Baseline and omitted controls

The reproduced baseline reaches **16,020 states and 60,238 transitions**. All nine invariant checks pass: debit conservation within the allowance; at most one local admission; at most one remote commitment; current context and required evidence at admission; pending retention until authenticated terminal evidence; commitment backed by admission and debit; retained pending coverage at renewal; consistency between consumption and admission; and release of the business reservation after observed terminal closure.

Table 6: Single-control omissions and shortest counterexamples

Omitted control	First violated requirement	Trace length
Required negative evidence	Current evidence at admission	4
Current-state comparison	Applicable authority at admission	4
Local logical-operation consumption	At most one local admission	5
Receiver idempotence	At most one remote commitment	6
Pending retention after cancellation request	Pending until terminal evidence	4
Permanent debit after closure	Debit conservation	6
Permanent debit across replacement	Debit conservation	3
Pending coverage during renewal	Retained continuation coverage	4

The failures distinguish mechanisms. Without current-state comparison, the trace issues a lease, generates a proof, revokes authority and admits the stale result. Without required negative evidence, an adverse record exists before proof generation, yet the weakened evaluator accepts. The first failure is stale applicability; the second is incomplete evaluation. A proof system that faithfully proves the weakened computation would not repair the latter.

Without local consumption, a second request obtains a fresh proof after the first admission and is admitted again. Without receiver idempotence, a duplicate remote delivery commits twice despite one local admission. Treating a cancellation request as closure removes pending exposure before the remote outcome is established. Refunding risk after confirmed cancellation violates debit conservation even though releasing the business reservation is legitimate.

The baseline also contains positive behaviour witnesses. Renewal adds its permanent debit while retaining pending work. Replacement preserves consumption, pending exposure and debit. A previously admitted remote effect can commit after revocation. Confirmed cancellation followed by an authenticated terminal receipt releases the reservation while preserving the assurance charge. These reachable traces show that passing the invariants is compatible with asynchronous work and controlled closure; the model does not achieve safety by refusing every operation.

6.3 Further counterexamples and validation limits

Two additional worked calculations illustrate scoped trust and concurrent renewal. A recognition path whose effective predicate scope is safety assessment cannot authorise task delegation; unrestricted transitive trust incorrectly accepts it. In the renewal example, two non-atomic readers both observe one remaining unit and jointly oversubscribe a two-unit ceiling. A compare-and-swap update accepts only one renewal. These are illustrative calculations outside the 16,020-state enumeration.

Four further examples compare explicit predicates for claim-type escalation, a management-system certificate substituted for system safety, a test result reused after configuration change, and component claims conjoined into system compliance without a composition rule. These

are small executable truth-table examples. They are not tests of an implemented general claim validator. The configuration example assumes no accepted conservative applicability mapping; an implementation must also test justified mappings rather than reject every identifier difference.

The finite checker in this section checks only the already issued lease branch. It does not check prospective proof-to-lease conversion, continuous clock semantics, residual-tail closure, distributed consensus or physical actuation. Appendix C separately tests selected prospective-context and integer-clock checks in concrete functions; it supplies neither exhaustive verification nor a cryptographic proof implementation. The analytical propositions state the obligations for those wider constructions. Finite state counts cannot establish computational soundness or deployment risk. Conversely, the shortest failures provide concrete regression fixtures for implementations and show why each selected control appears in the protocol.

6.4 A separate executable reference gate

The companion package also contains a plaintext evaluator and protected-state reference for selected functions that the finite checker abstracts. It processes typed records, compares complete committed source views, checks rooted recognition paths, validates claim scope and evidence predicates, and binds a permitting decision to current admission state. Twenty-nine deterministic scenarios pass. They include successful issued and prospective flows, refusal of incomplete or changed evidence, narrowed assessor scope, action substitution, clock bounds, repeated operations, renewal accounting and persistent pending obligations.

Appendix C specifies the fixture schema, scenario groups, results and implementation limits. These are function-level tests in a single process with trusted registries and in-memory state. They do not run OpenClaw, implement private proofs or establish distributed durability. They complement the exhaustive interleaving model without enlarging its state-space coverage. The distinction allows an engineer to reproduce the tested control behaviour while identifying the remaining implementation work.

7 Evidence mechanisms and practical assessment

7.1 From an assessment artifact to a usable claim

Cross-organisation assurance begins with a precise question: what does this evidence establish about the proposed operation? An evaluation report, a management-system certificate and an attestation result answer different questions. Their signatures may all be valid while only some are relevant to the requested action. The claim-validation relation preserves these distinctions through the assessed object, proposition, method, issuer competence, deployment context and validity contract.

Operational alignment concerns conformity to the declared objective, mandate and constraints at covered interfaces. A retrospective claim refers to recorded effects over a specified interval. A prospective claim concerns the controller and the assumptions under which it constrains future effects. Neither establishes the model's internal goals. A safety claim identifies the deployment, named harm, operating conditions and safety property or conditional risk bound. A compliance claim identifies obligations, jurisdiction, organisational role and the permitted assessment route. These claim classes can share evidence without becoming interchangeable.

Testing, evaluation, verification and validation (TEVV) results require an applicability argument. A result for one model revision and tool configuration cannot automatically support another deployment. The same applies to organisational evidence. ISO/IEC 42001 concerns an artificial

intelligence management system, whereas ISO/IEC 42005 concerns AI system impact assessment [33, 45]. A management-system certificate can support an assurance case about the supplier’s processes. The proposed operation still requires evidence about the relevant system, configuration and controls.

The European Union Artificial Intelligence Act further distinguishes regulated roles and assessment routes. Provider and deployer obligations depend on the applicable classification and context. Notifying authorities designate and supervise notified bodies, while market-surveillance authorities supervise and enforce compliance [42, 43]. Consequently, the protocol records the legal meaning of an assessment output. It verifies the source and permitted reliance on that output; the competent institution retains its legal powers.

7.2 Evidence produced by existing mechanisms

Table 7 relates available mechanisms to the questions they can support. These are documented tools and interfaces. Their combination within the proposed protocol is an architectural contribution, rather than a reported integration by the organisations concerned.

Table 7: Existing evidence mechanisms and their limits

Mechanism	Evidence object	Example question	Boundary
Inspect AI [47]	Evaluation logs containing task and model details, generation configuration, sample inputs and outputs, scores and run status	Did the tested agent follow the specified approval rules in the evaluation?	The result applies to the tested configuration and conditions; transfer requires justification.
AI Verify Foundation Project Moonshot [48]	Benchmark and red-team results for large language models and applications, complemented by process checks	How did the application respond to the selected unsafe requests or attempts to obtain protected data?	Tool use alone establishes neither assessor independence nor certification.
MLCommons AILuminate Safety [49]	Hazard-specific and overall ratings under a published assessment method	How did the named chat system handle the benchmark’s defined content hazards?	The cited v1.0 scope uses single-turn interactions across twelve hazard categories; it does not establish safety of a multi-step industrial agent.
CycloneDX model cards and machine-learning bill of materials [50]	Structured model parameters, dataset references, performance measures, limitations and component context	Does the deployed composition correspond to the composition described in the assessment?	Inventory accuracy and completeness need supporting evidence.

Continued on next page

Table 7 (continued)

Mechanism	Evidence object	Example question	Boundary
GitHub artifact attestations [51]	Signed provenance connecting an artifact with its workflow, repository, organisation and source commit, using Sigstore and the Supply-chain Levels for Software Artifacts (SLSA) framework	Did the admitted container originate from the accepted build process?	Build lineage and integrity do not establish behavioural safety.
Remote Attestation Procedures (RATS) and Project Veraison [52, 53]	Attester Evidence, appraisal inputs and a verifier’s Attestation Result	Does the requesting environment satisfy the declared appraisal policy before a protected key is released?	Coverage is limited to measured properties and policy. RFC 9334 is Informational; Veraison’s example Docker deployment is intended for development.

A usable evaluation package identifies the sampling procedure, population, evaluator version, scoring rules, acceptance criteria, uncertainty and exclusions. Seeds or repeated-run settings are retained where meaningful. Model-based grading additionally identifies the grading model and evidence of its validity. Inspect’s logs support examination of individual cases and run status; its score-edit mechanisms can retain the author, reason and earlier values [47]. Such detail permits scrutiny of a result that an aggregate score alone would conceal.

A benchmark failure fraction remains a statistic of the test procedure. Converting it into a probability of a named operational harm requires an additional argument about deployment conditions, exposure and the observation process. Likewise, replaying an evaluation successfully establishes reproducibility under its declared conditions; it does not establish that those conditions cover every operational situation.

Evidence often arrives as a report, log or registry entry. An adapter commits to the original artifact and maps selected fields into the shared claim profile. Where the adapter issues a verifiable credential, its signature attributes the extraction to the adapter. The original assessor remains separately identified. The relying verifier authenticates the source through an accepted mechanism and validates the mapping. This prevents an integrator from acquiring an assessor’s competence merely by signing a wrapper around the assessor’s report.

7.3 Assessment provenance and recognised competence

Actual assessment activity makes this distinction concrete. In March 2026, BSI Group reported ISO/IEC 42001 accreditation from the United Kingdom Accreditation Service (UKAS), the Dutch Raad voor Accreditatie (RvA) and the ANSI National Accreditation Board (ANAB) [34]. DNV’s December 2024 announcement records RvA accreditation for the same management-system standard and identifies Cognizant as its first customer receiving accredited certification [35]. These examples establish activity in a defined certification field. Reliance requires the actual certificate and current scope of its legal issuing entity.

Other outputs have different meanings. TÜV SÜD offers a voluntary AI Act Attestation of Conformity for a defined assessment scope. Its service description distinguishes that output from an EU Declaration of Conformity and states that the attestation itself does not enable CE marking [36]. UL Solutions announced certifications under UL 3115 for Qcells and Omniconn in March 2026 [37]. UL 3115 is an Outline of Investigation. A result about a product or component still requires an applicability argument for its intended installation.

For statutory notified-body claims, the verifier checks the legal entity, notification identifier, legislation, authorised tasks and current official record. The Commission’s New Approach Notified and Designated Organisations (NANDO) directory provides the relevant legislative scope [54]. Recognition of a commercial brand cannot replace these checks.

The trust path is therefore typed. A relying organisation accepts an accreditation authority for a specified scheme, verifies the certification body’s corresponding accredited scope, and checks the supplier’s certificate. The path supports recognition of that certificate issuer. A separate mandate path authorises the agent’s proposed operation. Neither path supplies the other’s missing premises.

7.4 Industrial illustration and developments in assessment practice

UL Solutions’ Qcells case provides a documented industrial comparison. It concerns an identified AI control component in a data-centre energy-management system. The system produces time-bound energy plans for operator approval before physical execution. UL describes documentation review, mapping supplied evidence to requirements, process auditing, witnessed or customer-performed testing, fault-related demonstrations and stress testing. Its assessment also covered evidence of approval gates, rationale, audit trails and rollback or restore capability [38]. The case demonstrates how design, oversight and operational evidence can be assessed together within a defined component scope.

Applying the proposed protocol to such a setting would bind the assessed component and installed configuration to a bounded energy-adjustment request. The operator’s mandate would determine approval authority, and the admission controller would evaluate current applicability before the equipment adapter admitted the effect. This is a proposed integration, rather than a reported Qcells implementation. A software change could leave the original certificate authentic while preventing reliance on it for the changed operation.

Emerging evaluation work increasingly distinguishes whole agent tasks from isolated model responses. MLCommons’ AILuminate Agentic Product Maturity Ladder separates capability, task boundaries, confidentiality, user control, robustness, security and reliability [55]. Its web-page example table is illustrative. The framework nevertheless gives a concrete reason to bind evidence to the task, tool permissions and operating context rather than a model name alone.

The AILuminate Global Assurance Program, announced in February 2026, sets out plans for benchmarking services, risk labels and regional or industry-specific assessment frameworks [56]. These stated aims indicate an emerging shared assessment infrastructure. They do not establish universal equivalence of results or demonstrate that every planned service is available. Reuse across buyers remains conditional on method, scope and local acceptance policy.

Assessor competence is also becoming more explicit. ISO/IEC 42006:2025 adds requirements for bodies auditing and certifying AI management systems against ISO/IEC 42001 [39]. A March 2026 bulletin from Deutsche Akkreditierungsstelle (DAkkS) and UKAS separately addresses AI used within accredited conformity assessment. It treats its introduction as a significant change requiring notification and retains institutional responsibility for final conformity decisions [40]. Accordingly, the protocol records both the assessor’s recognised scope and any relevant use of

AI in the assessment method. The deterministic controller validates the resulting record and applies admission rules; it does not assume the institution’s conformity-assessment role.

8 OpenClaw case study: protected engineering-data release

8.1 Why OpenClaw is a useful design case

OpenClaw provides a useful concrete setting for the distinction between an agent runtime and an authority to commit an effect. Its documented deployment model permits multiple agents within one Gateway, with separate agent state and configurable tool restrictions. The security guidance nevertheless assumes one trusted operator boundary per Gateway. For mutually untrusted participants, it recommends separate Gateways and credentials, preferably with separate operating-system users or hosts. Accordingly, organisation A and organisation B use separate installations in this case. Their relationship is governed by the cross-domain protocol, rather than inferred from agents sharing a runtime [57, 58].

Existing controls deserve explicit recognition. OpenClaw supports layered tool restrictions; later restrictive layers cannot restore tools denied earlier. Sandboxing moves supported tool execution into a configured environment while the Gateway remains on the host. It is optional and disabled by default. A workspace is not, by itself, an operating-system security boundary. The case therefore fixes and inspects the effective sandbox, tool, credential and network configuration, including overrides and elevated execution. A configuration declaration is evidence about configured controls, not proof that every protected effect is mediated [58–60].

OpenClaw also has substantive execution-approval mechanisms. For node execution, the current documentation describes a canonical plan recorded at approval and reused at dispatch; changes to command, working directory, agent or session cause rejection. Native approval remains relevant, but approving a command does not establish that a remote service’s safety evidence applies to a proposed data release. The proposed protocol adds those claim and continuation checks without replacing the local operator’s approval authority [61].

The current A2A channel implements authenticated task exchange using the A2A 1.0 JSON remote procedure call (JSON-RPC) binding. It separates sessions by authenticated peer and context, and remote tasks cannot resolve operator approvals through slash commands. These are useful ingress properties. The documented implementation also has important limits: reply timeout leaves work running, task cancellation is refused, and A2A task records are held in memory and lost on Gateway restart. These are properties of this implementation, not conclusions about the A2A standard as a whole [62].

The proposed addition is a protected resource gateway outside the planner’s authority. OpenClaw submits typed operation proposals to it. The gateway hosts the deterministic evaluator and serialised admission controller, while a resource-specific adapter controls the actual release or simulation dispatch. A plugin hook can provide early feedback and connect the workflow, but the effect adapter remains authoritative. OpenClaw documents that plugins run inside the Gateway process and that hook coverage depends on the runtime; registration alone is not evidence of complete coverage. Direct credentials or alternate routes to the protected resource must therefore be unavailable to the planner [63].

8.2 A protected engineering-data workflow

Organisation A owns a design file; organisation B offers a simulation service. A’s agent may request one simulation for a named engineering purpose. A’s mandate permits release of one

identified revision to one identified service endpoint, subject to an accepted assessment, current configuration, output restrictions and a bounded continuation. B’s OpenClaw agent coordinates workers but cannot enlarge that mandate. This is a proposed arrangement, not a reported OpenClaw deployment.

In tuple (1), A is the principal. The submitting instance and possession key identify A’s agent, the executor identifies the release adapter, and the recipient identifies B’s approved service. The resource and payload commitment identify the file revision. Typed parameters include purpose and processing restrictions. A protected allocator supplies the logical operation slot u ; the episode identifier e follows the exposure and its obligations across workers. The A2A message, task and context identifiers are correlation fields. None substitutes for u , a mandate or an executor identity.

A first requests evidence for the specific B deployment. Required records may include a signed build manifest, the effective sandbox and tool configuration, a relevant evaluation report, an assessment whose scope A accepts, and current incident and withdrawal records. A challenge fixes the policy, evidence partitions, query rules, configuration and freshness requirements. An agent-selected collection of positive reports is insufficient. B may keep sensitive evidence private, but the private relation must evaluate the complete declared view, including applicable adverse records.

Validation preserves claim type. A trace test showing that an agent obeyed a release restriction supports the tested operational-alignment claim. A sandbox test concerns that control under its stated conditions. An AI management-system certificate concerns the management system within its scope. None alone proves that the simulation service is safe, generally aligned or legally compliant for A’s use. A accepts only an explicitly justified inference for the exact deployment and operation; the assessor, source signer and evidence-wrapper issuer remain separate roles.

Table 8 illustrates a conditional validation record, not an actual supplier assessment.

Table 8: Illustrative ClaimValidationRecord

Field	Illustrative content
Source claim and assessed object	Claim <code>c-eval</code> ; B’s identified simulation-service artifact
Proposition	The required export-control tests satisfy the declared acceptance rule
Configuration and context	Committed model, container, tools, access rules and engineering-simulation purpose
Method and evidence	Versioned test procedure, scoring rule, report commitment and supporting manifest
Assessment provenance	Named source assessor; independent-assessment status only where substantiated
Validation issuer	A’s relying verifier; any extraction adapter identified separately
Recognition basis	A’s accepted assessor scope and relevant recognition-path versions
Validation result	Support under the declared evidence and acceptance premises

Continued on next page

Table 8 (continued)

Field	Illustrative content
Applicability and status	Required configuration mapping, snapshot versions, expiry and contradiction dependencies
Operation binding	Proposed release identifier and continuation reference; no execution permission issued by this record

The evaluator then decides on the pair (a, π) : this release and its qualifying continuation. The episode service issues the covering continuation and records its permanent debit before exposure. At admission, the controller rechecks current applicability and versions, verifies the unconsumed slot and the issued continuation, and durably records consumption, the reservation, pending obligation and outbox item. The resource adapter subsequently dispatches the bound operation. B’s adapter acknowledges acceptance against the same logical operation. A lost acknowledgement initiates reconciliation, not an additional release. Native approvals remain prerequisites wherever the local execution policy requires them.

8.3 Adversarial traces and the paper’s results

Consider an evaluation that permits release under configuration c_0 . An adversarial planner subsequently presents a different recipient, omits a blocking report or retries after a timeout. The following trace variants identify the relevant control rather than treating all failures as prompt injection.

Table 9: OpenClaw design-case traces and required controls

Trace variant	Failure without the stated control	Required Paper 3 response	Property illustrated
An applicable adverse record is omitted from the submitted evidence	A positive subset appears sufficient	Verify the required complete query; reject a proven blocker, or withhold support if completeness is unresolved	Evidence-view fidelity
Configuration changes from c_0 after evaluation	A historical permit is used for a changed deployment	Detect a changed dependency at admission; obtain a new applicable decision	Current applicability
The agent substitutes a wider path, audience or executor	Narrow approval becomes broader authority	Compare the canonical exact operation and recognised scope; reject the mismatch	Scope preservation
An acknowledgement is lost and the agent changes its task identifier	A retry creates a second release	Reconcile the durable slot u ; preserve its existing consumption and effect record	Replay resistance

Continued on next page

Table 9 (continued)

Trace variant	Failure without the stated control	Required Paper 3 response	Property illustrated
B has accepted work when A revokes further authority	A local stop is mistaken for remote closure	Block new ordinary admissions; retain pending exposure and invoke the specified containment response	Persistent obligations
A private proof covers an old accepted view	Proof validity is mistaken for current permission or factual truth	Recheck dependencies and admit separately; retain source-truth assumptions	Evaluation–admission separation

The separate reference gate supplies selected executable counterparts: RG02–RG05 check adverse-view completeness and updates; RG06–RG10 check deployment, action and issuer bindings; RG16–RG20 check prospective context and replay; and RG21–RG29 check pending retention, accounting and receiver behaviour. The original finite model supplies the asynchronous revocation and delayed-receipt traces. These results support the proposed control logic within those artifacts, not a claim that OpenClaw has been instrumented or tested.

These distinctions have concrete precedents. A maintainer advisory published on 11 September 2026 describes reusable execution approvals that failed to bind the working directory. The same command could consequently have different effects in another directory. A second advisory describes file-transfer standing approvals whose path interpretation or node display-name matching could widen authority. Both identify versions before 2026.8.1 as affected and 2026.8.1 as patched. They motivate exact context and stable identity binding; they do not demonstrate that the patched release remains vulnerable or that Paper 3 has tested it [64, 65].

A separate installer advisory describes a time-of-check/time-of-use issue in which a validated tools-root path could be rebound before the final write. The documented fix pins a canonical root; the advisory identifies 2026.3.8 as the fixed release. This illustrates the implementation-refinement obligation: a correct decision over a path string is insufficient if the effect adapter subsequently resolves that string to a different object. The analogy is structural and does not claim that a protocol-level hash alone repairs filesystem races [66].

Stopping is especially consequential. Suppose B accepts the simulation, A loses the response, and A subsequently withdraws authority. The continuation retains the admitted operation, pending exposure, containment duties and permanent risk debit. It may direct B’s adapter to revoke data access, stop a supported job or quarantine an output. Which response is available depends on the actual service contract. Closure requires authenticated evidence of the agreed terminal condition; it cannot be inferred from a stopped chat session. Exposure already incurred cannot be undone by cancellation. An unresponsive remote service therefore remains an unresolved obligation, not a successful kill.

8.4 Refinement obligations and validation status

The mapping shows where the paper’s conditional properties would apply. It does not establish them for OpenClaw’s complete implementation. The runtime already documents bounded TLA+ security models checked with TLC for selected paths and explicitly distinguishes them from full-code verification. At review time, its documentation reports that the separately maintained

model repository is unreachable and that reproduction instructions are unavailable. Those reported models are related work, not independently reproduced evidence for this case [67].

A credible prototype should pin the runtime and plugin artifacts, demonstrate that every covered effect reaches the protected adapter, and replay the six trace variants across restarts and concurrent workers. It should preserve evidence challenges, decision dependencies, exact approval bindings, consumed slots, outbox records and closure receipts. Cross-domain tests should include a recognised assessor with the wrong assessment scope and an authentic report for a superseded deployment. Successful tests would support the tested implementation obligations. They would not establish an operational harm probability, the truth of every accepted source, general AI alignment or conformity certification of the proposed protocol.

The refinement map is explicit: a typed proposal supplies evaluator input; an accepted challenge supplies the evidence dependency vector; a committed slot, outbox and lease transaction maps to admission; an adapter receipt maps to an effect observation; and verified terminal evidence maps to closure. A2A task completion and OpenClaw session termination have no unconditional mapping to closure. A restart must preserve the protected state even when the channel task cache is lost. A route whose effects bypass that state lies outside the claimed implementation guarantee. Appendix C exercises selected parts of this map in a separate reference gate. It does not run OpenClaw or validate its code.

9 Industry architectures and published safety practices

9.1 NVIDIA: architecture, enforcement and evidence

NVIDIA’s current agent infrastructure provides a concrete setting in which to locate the proposed protocol. The relevant components occupy different layers: the Enterprise AI Factory supplies a reference architecture; NeMo Agent Toolkit supports agent applications; NeMo Guardrails supplies configurable checks; OpenShell constrains execution; and NemoClaw packages agent onboarding and lifecycle management. NVIDIA’s Enterprise AI Factory guide describes persistent workspaces, skills, sandbox policies, evaluation and observability as components of long-running agent workflows [68]. This is architectural guidance and a set of implementation building blocks. It does not, by itself, establish that an external organisation should accept a particular safety claim or authorise a particular effect. Paper 3 supplies a candidate contract between these layers and the relying resource owner.

The review is version-bounded. OpenShell v0.1.1 was released on 26 September 2026; NeMo Guardrails v0.24.1 is dated 16 September 2026; NemoClaw’s release notes list v0.0.129 on 23 September and continue to identify the project as an early preview [69–71]. NemoClaw is an open-source reference stack, and its enterprise-readiness guidance explicitly distinguishes implemented functionality, operator responsibilities and roadmap work. It is not represented there as a hardened, multi-tenant enterprise control plane [72]. OpenClaw is an agent runtime within this discussion; NemoClaw supplies an integration around it, while OpenShell owns the sandbox boundary. These names should not be used interchangeably or treated as assurance certifications.

Protected enforcement and policy proofs

OpenShell makes the separation between an untrusted agent and protected enforcement concrete. Its gateway owns policy, provider and sandbox state; a trusted supervisor outside the workload mediates network access; and the workload’s other egress is denied. The documented launch

sequence binds the workload to its admitted sandbox and checks the isolation boundary before starting the agent. Linux filesystem and process controls complement the network boundary [73]. This is closely related to Paper 3’s separation of proposer, evaluator and protected controller. It supports the feasibility of the decomposition, while also narrowing the paper’s novelty claim: out-of-process enforcement and protected policy control are established implementation directions, not inventions of this protocol.

OpenShell also has a policy prover based on satisfiability modulo theories. It checks whether a candidate policy remains within a specified boundary and separately supports risk checks for proposed network rules. The documented containment model covers filesystem, process identity, Landlock, L4 connections and REST requests. Unsupported features, including GraphQL or MCP rules, produce an unsupported outcome; solver failure or timeout is not success [74]. A containment result is useful evidence for a bounded control claim. It does not establish the truth of every external assessment, the adequacy of the boundary policy, or a probability of operational harm. A Paper 3 adapter would record the candidate and boundary digests, prover version, coverage and outcome, and preserve these limits in the claim it issues.

Two extension surfaces offer plausible integration points. OpenShell gateway interceptors can validate selected control-plane writes and observe committed responses, while supervisor middleware can inspect and block supported network content before forwarding. The middleware blocks affected traffic on service failure by default, but cannot inspect every traffic form: exclusions include TLS-skipped endpoints and some response bodies. Gateway post-commit interception is observational and cannot reverse a committed operation [75, 76]. Therefore, placing a Paper 3 verifier at an available hook is only the beginning of an implementation. The deployment must show that every relevant effect crosses the chosen hook and that the final, possibly transformed request remains bound to the admitted operation. Resource-specific enforcement remains necessary wherever transport inspection cannot establish the complete business effect.

Guardrails, skills and operational boundaries

NeMo Guardrails contributes a different kind of control. Its current documentation describes tool allowlisting and JSON Schema argument checks, together with tool-result structure and linkage checks. It explicitly limits result validation to consistency within the submitted request: that check does not authenticate a remote execution result or establish cross-turn provenance [77]. The runtime-security guidance also leaves resource authentication and authorisation with the application or owning service. Rails may use rules, classifiers, custom actions or model-based judgments, and failure behaviour depends on the selected configuration [78]. A passed rail can consequently become evidence for its declared predicate; it should not be promoted automatically into an AdmissionReceipt, an EffectReceipt or a universal alignment claim.

This distinction refines the deterministic-controller concept. Model-based rails can supply observations or assessments, but the final eligibility evaluator must consume a fixed, versioned record of those results under explicit acceptance rules. The controller then checks current dependencies and performs the protected admission transition. Determinism belongs to that specified evaluation and state transition; it does not turn a classifier into a deterministic oracle about the world. For a required safety check, an unavailable provider, malformed response or unsupported scope must remain distinguishable from a substantiated rejection and from successful support. Reproducible policy processing and empirical detector quality are separate obligations.

NVIDIA’s emerging skill-verification tooling illustrates how usable evidence can be produced. Current documentation describes detached OpenSSF Model Signing signatures covering a skill directory, with a publisher certificate as trust anchor, and explicitly states that signing does

not prove the skill safe [79]. SkillEvaluator combines validation and security scanning, semantic comparison, and live sandbox evaluation with and without the skill. Its published benchmark records include the tested agents, evaluation date and measured dimensions; missing task sets cause live evaluation to be skipped [80]. These artifacts can populate Paper 3’s subject, method, evidence and applicability fields. Recognition of NVIDIA as the publisher authenticates origin within that accepted trust path. It does not independently confer accredited assessor status, authorise a downstream data release or establish suitability for another model and tool configuration.

For the Section 8 workflow, the NVIDIA stack could produce skill digests, benchmark conditions, policy-containment results and configured-control records. Paper 3 would bind those records to a particular release and continuation, including required adverse-evidence queries and current dependencies. The reviewed components do not establish the proposed private evidence relation or cross-organisation accounting and closure contract. NemoClaw also assigns cost controls elsewhere [72]; financial quotas would remain distinct from assurance debits even if implemented. The integration must demonstrate each boundary rather than infer assurance from a signed skill or a passed rail.

The resulting research programme is implementation-specific and testable. It should pin releases and effective policies, demonstrate complete mediation for the chosen effect class, add authenticated evidence and receipt adapters, and test replay, transformed requests, stale evidence, revocation, lost acknowledgements and worker replacement. NVIDIA’s documented lifecycle trust boundary, which treats writable agent state as untrusted and serialises host mutations, supplies a useful comparison for that refinement work [81]. The historical Safety for Agentic AI Blueprint should be cited cautiously: its catalogue page is now marked deprecated [82]. Current components and their stated limits provide stronger grounding than an undifferentiated claim that a vendor’s safety stack already implements cross-organisation assurance.

9.2 Evidence status of provider practices

Published provider practices offer concrete implementation patterns and evidence about their limits. Their evidential status varies. Product documentation describes supported mechanisms. Engineering reports describe the provider’s experience with particular deployments and experiments. Frontier safety policies state organisational procedures and commitments. This paper uses each source within that scope. The protocol proposed here has finite-model validation and bounded reference-gate tests; it has not undergone a production field trial. The practical guidance below therefore combines **published implementation guidance**, selected provider-reported operational findings, and additional controls derived from our formal model. Its effectiveness must be evaluated in the target deployment.

9.3 OpenAI: action boundaries and evidence

OpenAI’s March 2026 analysis of prompt injection treats the problem as an interaction between untrusted information and consequential capabilities. It recommends limiting the effects of manipulation even when the model accepts an attack. Its examples include deterministic transaction limits and controls over transfers to third parties. This supports the separation between probabilistic proposal generation and a protected admission boundary [83]. Our additional requirement is precise binding: the action descriptor must identify the recipient, resource, operation, payload, quantities and continuation. Inference from a plausible user objective is insufficient to establish that complete mandate.

OpenAI’s agent-safety documentation recommends structured data between workflow nodes, separation of untrusted input from privileged instructions, tool approvals, input checks and evaluation of execution traces. These are useful implementation measures, although the cited Agent Builder page is now part of legacy documentation [84]. Schema validity does not establish that a claim is true or that its issuer is competent. In this protocol, structured outputs feed typed claim validation; they cannot directly write the authority graph or risk ledger. A human approval, when required, becomes an attributed, scoped and expiring input to the same decision process.

The current agent sandbox documentation separates the harness from execution compute. The harness retains orchestration, authorisation, approvals, auditing and recovery, while the sandbox executes model-directed work with limited mounts and credentials [85]. This maps closely to our control architecture: a pure evaluator, a protected serialised admission service and an effect adapter. The mapping is architectural, not a claim that the SDK implements our semantics. Keeping the admission ledger outside the worker also preserves evidence and pending obligations when that worker crashes, is replaced or is retired.

OpenAI reports an operational cycle in which automated adversarial testing exposed attacks against Atlas, informed model training and strengthened surrounding safeguards [86]. This motivates repeated testing of the whole execution path. Paper 3 adds freshness dependencies: a material change in a model, connector, policy, evaluator or assessment can invalidate the evidence on which an earlier permission depended. Historical success on an attack suite remains useful evidence, but its applicability must be re-established for the changed configuration.

9.4 Anthropic: containment, consent and classifiers

Anthropic’s engineering guidance distinguishes predefined workflows from agents that choose their own steps. It recommends simple compositions, clear tool interfaces, environmental feedback and explicit stopping conditions [87]. These choices reduce the number of transitions an assurance argument must cover. Dynamic workers still need individual instance bindings, inherited scope restrictions and a common episode account. Delegating a task does not transfer the parent’s full authority or erase its outstanding obligations.

Anthropic’s sandbox implementation combines filesystem and network controls. Its cloud Git proxy holds the upstream credential outside the worker and checks the repository interaction before attaching that credential [88]. This provides an implementation pattern for a protected effect adapter. Our protocol extends the check to a cross-organisation operation contract and current evidence state. Domain allowlists alone do not supply this precision. Anthropic describes a disclosure in which a permitted provider domain accepted a file upload into an attacker’s account. It also reports defects caused by parsing or executing project configuration before the trust decision [89]. Those examples motivate validation of the actual tenant and recipient, and a trust boundary established before configuration can acquire execution authority.

Classifier-based approvals need a separate evidence category. Anthropic reports measured classifier errors and consent-scope failures [90]; the appendix records dataset sizes and metrics. A classifier may flag suspicious behaviour or provide uncertain evidence to Paper 2’s supervisor. It should not manufacture a mandate or replace Paper 3’s deterministic checks of typed, authoritative records.

9.5 Governance and independent assessment

OpenAI’s Frontier Governance Framework describes risk assessment, safeguards, incident response and external expert input for serious frontier-model risks. It identifies the Preparedness Framework as its underlying approach [91]. Anthropic’s Responsible Scaling Policy version 3.0 separates company plans from industry-wide recommendations, adds risk reports and frontier safety roadmaps, and expressly treats roadmap goals as goals rather than hard commitments [92]. These publications can support claims about a provider’s governance processes. They do not by themselves attest that a customer’s particular action, system composition or legal obligation set satisfies this paper’s eligibility rule.

OpenAI’s September 2026 assessment principles emphasise claims with explicit conditions, evidence-supported safety cases, independent scrutiny and testing of safeguards under realistic conditions [93]. This closely matches our distinction between authenticity, competence, applicability, method and evidential support. Our profile binds these distinctions and local acceptance rules to an exact operation, continuation and current admission state. Confidential verification can reduce disclosure while preserving an assessment’s stated scope. It cannot replace an assessor’s access to evidence needed to challenge a safety argument, nor can a credential wrapper convert a provider statement into an independent assessment.

OpenAI’s September misalignment-reporting framework also makes negative evidence relevant throughout training, evaluation and deployment. It publishes individual observations without presenting them as frequency estimates [94]. A corresponding implementation should connect incident records to the affected claims and configurations. An unresolved contradiction can then change a local validation outcome or suspend new admissions. Existing remote effects remain in the pending register until the resource supplies the required closure evidence. This connects disclosure and reassessment to enforceable operations rather than leaving them as disconnected reports.

Table 10: Published provider practices and the proposed integration

Control objective	OpenAI published practice	Anthropic published practice	Paper 3 integration
Protect the decision boundary	Keep trusted harness state separate from sandbox compute [85].	Keep upstream credentials in a validating proxy [88].	Place evaluation, serialised admission and effect mediation in distinct protected components.
Limit the actual effect	Apply source–sink controls to potentially sensitive transfers [83].	Check more than a permitted domain: account and operation also determine exposure [89].	Bind recipient, tenant, resource, payload and typed limits.
Preserve the trust status of input	Separate untrusted text and use constrained intermediate data [84].	Establish trust before project configuration can execute [89].	Accept claims through typed provenance and competence validation.
Make autonomy bounded	Use scoped requests and consequential-action confirmations [86].	Combine sandbox boundaries with selective approval mechanisms [88, 90].	Represent approval scope and validity explicitly; evaluate the admitted continuation.

Continued on next page

Table 10 (continued)

Control objective	OpenAI published practice	Anthropic published practice	Paper 3 integration
Test and revise safeguards	Use adversarial workflow testing and independent assessment [86, 93].	Measure both classifier misses and unnecessary blocks [90].	Version evidence, test freshness races and update affected claims after changes.

10 A working framework for AI safety engineers

10.1 Convert the safety argument into an operation contract

Begin with one consequential effect and one named harm. For the case in Section 8, the effect is release of a design revision to a specified simulation tenant; the harm is disclosure outside the declared processing boundary. Identify the principal who may permit the release, the resource owner who can enforce it, the assurance owner who maintains the continuation model, and the evidence owner who can answer a challenged claim. One team may fill several roles, but the records must preserve the distinctions.

Write a contract containing the action fields in (1), the intended deployment, permitted purposes, business limits, evidence requirements, ordinary expiry, protective actions and closure criterion. Inventory every route to the effect: tools, shell commands, direct API access, plugins, remote workers and recovery jobs. Assign an enforcing adapter to each covered route. A route with no enforceable boundary is a limitation of the safety claim. Record it before choosing a proof format.

Next, separate three reviews. The mandate reviewer decides what the principal authorises. The assurance reviewer evaluates the joint continuation and its model assumptions. The relying verifier validates the accepted evidence and its applicability. A model-generated explanation can help a reviewer locate evidence; it cannot replace an attributed decision or create authority. The release gate requires all three reviews to concern the same operation and continuation.

10.2 Implementation checks and retained evidence

Table 11 is a review checklist, with an observable test and a retained artifact for each control. Checks 1–8 are informed by published provider implementations and operational experience. Checks 9–12 are additional requirements of the proposed protocol. The complete checklist has not been field-validated as a single framework. The case tests in Appendix C evaluate a narrower executable implementation. Teams should record an owner, outcome, artifact digest, tested version and unresolved limitation for each applicable row.

Table 11: AI safety engineering checklist

Check	Implementation and verification action	Evidence to retain	Basis
1. Separate proposal and authority	Run model-directed code with no write access to admission policy, credentials, ledger or recovery state. Attempt each access from the worker.	Isolation configuration, negative test trace, controller version.	[85]; Paper 3 architecture.
2. Constrain each effect	Specify resource, tenant, recipient, operation, payload and quantity. Test an upload to a different account on the same permitted domain.	Operation contract, destination-resolution record, rejected request.	[83], [89]; Paper 3 binding.
3. Establish trust before loading configuration	Treat project hooks, plugins and startup settings as untrusted inputs until the applicable trust policy allows them.	Startup-order review and malicious-config test.	[89].
4. Combine filesystem and network controls	Limit mounts and egress; keep upstream credentials in a protected broker. Test direct and subprocess access.	Sandbox and broker policies; bypass-test results.	[88].
5. Keep instruction provenance	Separate external content from privileged instructions. Validate narrow typed outputs before passing them to a tool or evaluator.	Dataflow map and injection-regression cases.	[84].
6. Scope human approval	Show the exact resource, effects and disclosure before approval; bind the decision to that version of the action. Test recipient and payload substitution.	Approval record and mismatch rejection.	[84], [90]; Paper 3 extension.
7. Measure classifier error locally	Use representative benign, misuse and adversarial cases. Report datasets, false negatives, false positives and changes across versions separately.	Versioned evaluation report and labelled traces.	[90].
8. Exercise long workflows	Test poisoning of files, tool returns, intermediate state and worker handoffs; repeat after material changes.	Reproducible attack traces, corrected versions, regression outcomes.	[86], [87].

Continued on next page

Table 11 (continued)

Check	Implementation and verification action	Evidence to retain	Basis
9. Validate the assessment, not only its signature	Check assessor role, recognised competence, claim scope, method, configuration, time, status and contradictions.	ClaimValidation Record and scoped recognition path.	Paper 3; [93] motivates independent scrutiny.
10. Test the admission race	Change a relevant denial or revocation between proof generation and admission. Require rejection or recomputation before dispatch.	Ordered state-version trace and admission result.	Paper 3 current-state property.
11. Preserve pending obligations	Kill or replace the worker after remote acceptance, lose the receipt and retry delivery. Preserve the episode debit and reconcile through the resource’s idempotency key.	Durable pending record, deduplicated receipt, ledger history.	Papers 2–3 accounting and closure model.
12. Rehearse incident containment	Suspend new admissions, revoke dispatch authority, request cancellation and identify outstanding remote effects. Assign a human owner for unresolved closure.	Exercise timeline, acknowledgement evidence and remaining obligations.	Papers 2–3; [94] supplies a disclosure pattern.

10.3 Release, change and incident gates

Before enabling real effects, execute the reference workflow using synthetic data and a test resource. Require a valid operation to succeed as well as invalid operations to fail. Attempt recipient substitution, scope widening, an unknown claim type, a superseded assessment and a withheld blocking record. Demonstrate that the worker cannot write the ledger or retrieve the adapter’s credential. Capture the effective configuration, rather than only a template. An unsupported required rule or missing current record must stop ordinary admission with a typed reason.

Before increasing autonomy, test interleavings at the enforcing boundary. Pause between evidence validation and admission, then change authority, configuration or adverse evidence. Race two admissions for the same slot and two renewals against the remaining allowance. Drop acknowledgements, repeat delivery, restart the controller and replace a worker. Check the stored operation, episode, debit and pending set after each step. A database transaction supplies only its actual transaction boundary; a second service requires an explicit coordination and recovery argument.

On each material change, compare model, tools, sandbox policy, proof relation, evaluator, schema, issuer scope and assessment method against the dependencies of existing decisions. Invalidate affected reliance and obtain an applicable replacement before new ordinary admission.

Preserve the prior record for audit. A rollback also needs an explicit state-generation check: restoring yesterday’s database must not resurrect a consumed slot or a revoked permission.

During an incident, suspend new ordinary admissions at the resource boundary, preserve the ledger, identify admitted remote work and invoke its protective continuation. Contact the designated remote operator through the agreed process where technical cancellation is unavailable. Assign unresolved closure to a named operational owner. The incident remains open for that obligation until the contract’s terminal evidence is obtained. A closed chat, expired worker or revoked credential alone is insufficient.

10.4 Measurements that answer engineering questions

Report admission latency by stage: acquiring evidence, validating claims, producing and checking a proof or assertion, reading current status, serialising state and dispatching. Publish configuration and workload sizes. Report the delay from a relevant change to blocked new admission, the age of unresolved effects, and recovery behaviour under each injected fault. Distinguish blocked new work from already admitted work that remains in progress.

For behavioural detectors, retain separate false-positive and false-negative measurements, dataset provenance and the tested deployment. For private verification, record disclosed fields, query volume, output distinctions and timing leakage. For the effect adapter, measure duplicate suppression and reconcile all test operations with resource receipts. No universal threshold is proposed: deadlines and acceptable residual exposure must follow the actual harm contract and the calibrated construction of Paper 2. A weighted checklist score would conceal a failed mandatory boundary and should not replace these acceptance conditions.

The review packet should therefore contain the operation contract, authority path, claim-validation records, evidence dependencies, continuation and debit record, bypass and fault traces, effect receipts and outstanding closure obligations. This makes the decision inspectable by a second organisation while allowing it to apply a different acceptance policy.

11 Synthesis and differentiation of Papers 1, 2 and 3

The three papers study different failure mechanisms in one controlled workflow. Paper 1 asks whether cooperation remains a rational choice after computation costs and revocable authority enter a bounded program game [1]. Paper 2 asks when an authorised action has a supported continuation under partial observation and a persistent harm allowance [2]. Paper 3 asks how another organisation can verify the evidence supporting that decision, including private evidence, and attach the result to current admission and unresolved effects. Their shared object is an exact mediated operation. Their guarantees concern different properties of that operation.

11.1 Research questions, methods and evidence

Table 12 distinguishes the formal objects and evidence. The reported counts describe separate executable abstractions. They cannot be added to produce a combined coverage or safety measure.

Table 12: Distinct contributions and research directions

Dimension	Paper 1	Paper 2	Paper 3
Research question	Which cooperative program profiles survive verification costs and authority revocation?	Which action and continuation satisfy a declared episode harm bound under incomplete observations and changing workers?	How can organisations privately verify scoped evidence and preserve its applicability through admission and execution?
Formal object	A finite program game and an epoch-bound controller for atomic digital commitments	A finite bank of joint partially observed models, action-bound leases and a permanent episode account	Versioned evidence views, typed claim derivations and a protected admission and effect protocol
Method	Exact payoff comparisons; discrimination and transfer bounds; inductive invariants; finite exploration	Bayesian filtering within each retained model; conservative reachability; adaptive-renewal proof; synthetic enumeration; finite control checks	Bounded evidence semantics; recognition and mapping rules; conditional fidelity and refinement arguments; exhaustive control exploration
Executed checks	38,880 exact rational combinations; controller instances with 1,427 and 1,819 states; five control-omission counterexamples	432 synthetic scenarios and 120,060 terminal observation histories; largest controller instance with 24,448 states; twelve control-omission counterexamples	16,020 states and 60,238 transitions; nine invariants; eight control-omission counterexamples; 29 separate functional reference-gate scenarios
Decisive premises	Declared strategies and payoffs; current authority; sound checking; complete mediation and atomic covered effects	Containing joint model bank; sound inference; declared selection information; covered containment and pending effects	Accepted source and competence policies; faithful evidence computation; coherent current state; protected persistence and resource-specific effect mediation
Engineer-facing artifact	Incentive comparison and exact-action authority contract	Harm model, observation contract, continuation witness and charged lease	Evidence challenge, claim-validation record, proof or assertion, admission receipt and closure evidence

Continued on next page

Table 12 (continued)

Dimension	Paper 1	Paper 2	Paper 3
Research direction	Cost-aware implementation, richer deviations and incentive effects of intervention	Deployment calibration, scalable joint models and less conservative renewal accounting	Interoperable claim profiles, verified evaluator and proof implementation, distributed refinement and privacy evaluation

Paper 1’s analytical result separates active-game value, completion before revocation and implementation cost. The same external revocation process can favour a faster deviation. Its controller proof establishes specified authority, budget, binding, delegation and replay invariants independently of how proposals are generated. These are different conclusions: arbitrary proposal sequences can preserve the controller invariant without satisfying a strategic equilibrium. The paper’s private-interface bound also explains why an interface whose complete available observations conceal every strategically relevant distinction cannot support a response that requires that distinction [1].

Paper 2 addresses a different uncertainty. The supervisor can enforce the intended interface while remaining uncertain about the operating state. It therefore retains a joint posterior, including possible undetected prior harm, shared dependencies and queued effects. Its renewal theorem charges each full continuation before execution. Under its assumptions, the resulting first-harm bound retains the operational allowance plus any separately justified model-coverage error. Its numerical results come from synthetic enumeration. They provide neither a deployment hazard estimate nor evidence of general model alignment [2].

Paper 3 makes the evidence and reliance boundary explicit. An accepting private proof can establish that the prescribed evaluator accepted committed inputs. Scoped recognition can establish that a source is accepted for the relevant claim class. Protected admission can establish that the result applied at the authoritative transition. None of these mechanisms establishes the factual truth of a false measurement, the adequacy of a harm model or an assessor’s competence beyond its recognised scope. Institutional reliance and cryptographic verification remain separate premises.

11.2 One workflow, three distinct obligations

Consider the engineering organisations used throughout the programme. They first book compute capacity, then release a protected design chunk to a named simulation job. Their evidence contains commercially sensitive test reports and service details.

Booking. Paper 1 supplies the comparison between the joint booking and an individual fallback. The principal specifies permitted alternatives, timing and costs. The booking controller checks authority and the aggregate monetary budget at the atomic booking boundary. A successful booking creates capacity rights. It supplies no assurance about the subsequent release or the simulation service’s isolation.

Joint bounded release. Paper 2 introduces a separate release operation and the named harm of disclosure outside the declared isolation boundary. The supervisor updates its joint belief about service and worker state using available observations, while retaining queued work and the modelled containment behaviour. It selects a supported continuation and permanently debits

the exposure episode. A new worker or replacement retains that episode. The monetary booking budget and the risk account remain distinct quantities.

Private cross-organisation reliance. Paper 3 lets the relying organisation request a precise evidence relation without requiring unrestricted disclosure of the underlying reports. The challenge binds sources, adverse evidence, claim classes, configuration, operation and continuation. A private proof or accepted signed assertion answers that challenge under its declared trust assumptions. The protected controller then checks current applicability, consumes the persistent operation slot and records pending exposure before dispatch. Cancellation remains distinct from verified closure.

This workflow requires consistency across the interfaces. The same exact operation and compatible continuation must satisfy mandate, assurance and evidence-validation requirements. A management-system certificate may support an organisational-control premise; it cannot silently become a deployment-specific safety claim. Likewise, an authenticated evaluation result may support an observation or method premise without establishing a calibrated harm rate.

Private information requires particular care. An agent that sees confidential telemetry and selects a proposal using it can alter the information carried by the proposal itself. Hiding that telemetry inside a valid proof does not preserve Paper 2’s posterior automatically. The selection information must enter the declared observation model, be represented through its likelihood, or satisfy the paper’s explicit conditional-independence restriction. Proof generation and network waiting must also fit the charged continuation because they can extend exposure.

The combined engineering target is therefore a chain of discharged obligations, rather than a single general alignment certificate. Paper 1 contributes incentive and mandate discipline. Paper 2 contributes a conditional quantitative continuation claim. Paper 3 contributes scoped evidence reliance and current-state execution binding. Combining them requires an implementation refinement for the control properties and a stochastic coupling or conservative dominance argument for the harm probabilities. Strategic cooperation, calibrated safety and justified institutional reliance retain their own validation tasks.

11.3 Shared terminology and preserved distinctions

Table 13 carries the common vocabulary forward from Papers 1 and 2 [1, 2]. It distinguishes computation, authoritative state change and resource effect. The decision kernel is a bounded evaluator; transition and observation kernels in Paper 2 are probability laws. Reusing the word *kernel* does not make these objects interchangeable. Likewise, operational alignment means conformance to the declared mandate and supported continuation, not a conclusion about hidden model values or general AI alignment.

Table 13: Shared terms across the research programme

Term	Lineage in Papers 1 and 2	Meaning retained in Paper 3
Decision kernel / evaluator	Paper 1’s narrow policy checker is separate from state control.	The pure eligibility evaluator computes a fixed profile over explicit inputs. It does not consume authority or commit effects.

Continued on next page

Table 13 (continued)

Term	Lineage in Papers 1 and 2	Meaning retained in Paper 3
Protected state controller	Paper 1’s authority and resource state machine; Paper 2 adds assurance guards.	It admits exact operations against current state and preserves consumption and pending obligations. The three-function control architecture is the wider grouping.
Assurance supervisor	Paper 2’s joint belief, continuation evaluation and episode-account construction.	It supplies the quantitative assurance decision. Evidence validation does not replace inference, model coverage or calibration obligations.
Principal / instance / exact action	Paper 1 binds delegated authority to represented principals, admitted instances and covered actions. Paper 2 binds the next action and continuation.	Principal, instance, possession key, executor, recipient, payload and logical operation slot remain distinct bindings. A new task or instance does not create another entitlement.
Admission / commit	Paper 1’s Admit admits an instance; Commit creates its atomic covered effect.	Admit consumes an operation slot locally. Dispatch and remote effect commitment remain separate events. Similar names do not establish refinement.
Certificate / proof / lease	Paper 1 records a checked computation; Paper 2 issues a lease for a charged continuation.	EligibilityProof, LeaseRecord and EffectReceipt establish different claims. A checked computation record is not third-party conformity certification.
Exposure episode / continuation	Paper 2 retains one exposure episode through renewal, retries and worker changes; a continuation includes monitoring and containment.	Evidence, mandate and assurance must bind compatible joint behaviour. Task completion or worker retirement does not end the episode while covered exposure remains.
Business resources / risk allowance	Paper 1 bounds committed and reserved resources; Paper 2 permanently debits continuation risk.	Business reservations can be released under their contract. Issued risk debits persist through closure, renewal and replacement.
Epoch / generation / version	Paper 1 uses authority epochs; Paper 2 distinguishes decision epochs and assurance generations.	Qualify each field: authority epoch, decision tick, assurance generation, evidence version or configuration version. None substitutes for another.
Pending effect / containment / closure	Paper 2 retains unfinished exposure and distinguishes a stop request from effective containment.	The adapter returns typed observations. Unresolved effects stay attached to the episode. Cancellation or a terminal communication task alone does not establish closure.

Notation is local to each paper. Paper 1 uses ε for an equilibrium tolerance measured in utility units, whereas Paper 2 and the inherited assurance construction use ε for a first-harm probability allowance. Paper 1 uses λ as a cryptographic security parameter; Paper 2’s teaching example uses λ as a hazard rate. Paper 1’s e denotes an authority epoch, while e in (1) denotes an exposure episode. These symbols must be qualified when records or arguments cross the paper interfaces; none expresses an equality between the underlying quantities.

12 Deployment and implementation requirements

12.1 Placement of authority and supervision

A frontier-model provider may operate inference while its customer controls tools and protected data. The customer can enforce the proposed protocol at those tool and data gateways without inspecting model weights. Claims about provider-internal testing or safeguards remain external assertions with declared evidence and issuer scope. A private proof of a gateway decision establishes only the relation that the gateway verifies. Unmediated provider-side effects require another boundary and cannot be covered by assumption.

An independently operated model gives its operator more access to configuration and runtime measurements. It does not automatically provide complete mediation. Shell access, direct network routes, plugins and background workers must be included in the effect inventory. In a cloud–edge deployment, remote queues, clock uncertainty and intermittent connectivity affect admissibility and containment. A disconnected node must follow its pre-authorised continuation or decline new ordinary effects according to its lease; it cannot silently extend freshness or authority.

Micro, meso and macro supervision describe scopes. A micro controller mediates one effect boundary. A meso supervisor coordinates a workflow and its workers. A macro governance service manages acceptable roots, policies and aggregate constraints. The probability construction still requires a coherent episode account and a justified joint model [2]. Stacking three local bounds does not create a system bound when their failures share a provider or sensor. Hierarchy is useful only when obligations and dependence are preserved at its interfaces.

Temporary workers receive narrowly scoped, expiring bindings to the principal, episode and permitted operation set. Creating a worker does not create a new budget. Joining a worker changes the covered configuration and may require renewed evidence and a charged continuation. Retirement revokes future ordinary authority while retaining queued effects, consumption records and pending duties. An adversary who can create new task or agent identifiers must still encounter the same resource-consumption and episode-accounting scope.

12.2 Three feasible evidence modes

The first implementation can use an assertion profile: an accepted assurance service signs its result for the exact context, and the local controller validates attribution, scope, status and applicability. This is feasible with established signatures and protected state. Its trust premise includes the service’s calculation. A selective-disclosure profile can reduce disclosed credential fields while preserving those requirements. The reviewed BBS cryptosuite supplies relevant mechanisms but remains a Candidate Recommendation Draft [18].

A private-computation profile additionally proves the bounded relation over committed evidence. Its feasibility depends on the private source volume, canonicalisation and signature costs, query language, proof system, setup assumptions and action deadline. The complete-source baseline is deliberately simple and may be expensive. Authenticated range-query constructions, verified

preprocessing or accepted source assertions are optimisation options with different proof and trust obligations. No latency, proof-size or cost claim is made here because no end-to-end circuit is implemented or benchmarked.

The modes must remain distinguishable in records and negotiation. An application can permit an assertion mode for one claim and require private computation for another. Such composition needs an explicit assurance case. The strongest evidence mode for one prerequisite cannot compensate for missing authority or an unsupported prerequisite elsewhere.

12.3 Conformance and implementation gates

A protocol conformance suite should test semantic agreement and state transitions as well as message syntax. Required cases include unsupported extension refusal, exact-action substitution, stale status, time-boundary expiry, omitted denial records, scope expansion, unapproved claim promotion, changed deployment, duplicate delivery, lost acknowledgement, replacement during pending work and renewal races. Implementations must produce compatible outputs for published positive and negative fixtures under the same profile versions.

Appendix C supplies an initial plaintext reference gate for selected claim and admission functions. A production evaluator still needs a validated external parser, complete bounded profile semantics, approved maps and the prescribed outcome aggregation. Its controller needs protected durable recovery, serialised lease installation and effect-specific closure adapters. A later private implementation must establish the stated compiler relation and cryptographic assumptions. Two independently implemented gateways can then test interoperability under faults and changed evidence. The minimum demonstration combines an evaluation log, signed build provenance and a scoped assessment record, using clearly identified test issuers where real issuers expose no machine-readable interface.

Evaluation must report the complete path to effect admission: source acquisition and freshness, graph construction, validation, proof generation and verification, status checks, state serialisation and dispatch. It must also report disclosure, common dependencies, invalidation delay, recovery behaviour and containment latency. A fast proof with a stale registry or slow revocation path does not establish timely control. These measurements and field calibration are future work, rather than implied results of the finite exploration.

13 Discussion and limits

The specification places private evidence inside an explicit reliance and execution contract. Its fidelity result is relative to accepted source snapshots and a fixed evaluator. Its scope result is relative to accepted roots and adequate mappings. Its admission result is relative to authoritative ordering, freshness and clock assumptions. Its accounting result preserves recorded continuation charges and obligations. Each boundary names an external premise that a deployment must substantiate.

A malicious or mistaken accepted issuer can supply authentic false evidence. A source can omit an event from its own committed registry. An assessor can apply an inadequate method. A controller can faithfully enforce a policy that neglects a relevant harm or stakeholder. These failures require source governance, independent scrutiny, method validation and policy revision. Additional signatures or private proofs do not remove them. The protocol makes such reliance inspectable without claiming to settle all of it technically.

Private verification can also reduce visibility needed for independent oversight. A relying domain must be able to understand the proposition, method, scope, accepted dependencies and appeal route even when raw evidence is withheld. Public inputs, status lookups, operation timing and repeated binary decisions can reveal sensitive facts. Selective disclosure and zero knowledge address only their defined leakage models. Retention, purpose limits and query governance remain part of the deployment design.

The distributed boundary limits guarantees. A2A communication supplies no cross-organisation atomic commit. A coherent ordering requires a correct implementing service or protocol, with explicit behaviour during partition and recovery. A sender cannot ensure remote idempotence without a protected receiving boundary. An unobservable physical process may continue after a cancellation. A proof of control invariants therefore supports a conditional assurance case; it is not an unconditional safety certificate.

The finite exploration is intentionally small. It demonstrates that particular omissions break the selected invariants and that the stated bounded model excludes those traces. The separate reference gate tests selected evidence and admission functions with deterministic fixtures. Neither establishes parameterised verification, general semantic validation or a cryptographic implementation. The analytical propositions extend beyond the enumerated parameters under their stated assumptions, while implementation refinement remains an obligation for a concrete system. Publication of a private prototype, interoperable claim profiles and deployment measurements would supply the next forms of evidence.

The main research result is the explicit connection among three questions: what a private decision proves, whose claims it may use, and when it may authorise an effect. The proposed extension keeps those answers attached to one operation and continuation as evidence, authority and participants change. This provides a testable basis for bounded cross-organisation autonomy while preserving the distinction between operational conformance and the adequacy of the safety and compliance claims on which it relies.

A Proposed record schema

The following is schematic A2A 1.0 Message content. The example namespace is a placeholder for the research extension; the identifiers denote an illustrative operation. The outer request must separately activate and acknowledge the extension through its binding. This fragment is not an interoperable production schema.

```
{
  "messageId": "msg-17",
  "role": "ROLE_USER",
  "extensions": ["https://example.org/eligibility/v1"],
  "parts": [{
    "mediaType": "application/json",
    "data": {
      "type": "AdmissionRequest",
      "operationId": "op-17",
      "episodeId": "episode-4",
      "contractDigest": "sha256:...",
      "actionCommitment": "...",
      "leaseId": "lease-2",
      "proofDigest": "sha256:...",
      "expectedStateVersion": 31
    }
  ]
}
```

A complete schema additionally binds actor, possession key, executor, recipient, resource, payload, typed quantities, configuration, semantic and mapping versions, accepted roots, query scope, trust-policy version, challenge, validity interval and continuation position. A mutable URL is not an evidence identity. Referenced artifacts require digest binding and controlled retrieval. **expectedStateVersion** requests a comparison; only the resulting AdmissionReceipt records a successful authoritative transition.

The proof relation and record serialization are versioned together. Breaking semantic changes require a different negotiated profile. Canonical encodings must distinguish absent fields from empty values and specify integer ranges, units, byte encodings, domain separation, identifier comparison and error handling. A deployment must publish fixtures covering both accepted and refused records.

B Reproducibility and scope of the executable artifact

The accompanying reproducibility package contains the manuscript source, figure sources, the exact finite checker, the separate reference gate, their generated results and READMEs describing the assumptions. The checker uses the Python standard library. Running **python3 check_protocol.py** in its validation directory regenerates **results.json** and prints the baseline, ablation and worked-example summaries. Breadth-first search makes each reported ablation trace shortest under the stated enabled-event relation.

The package preserves the difference between implemented checks and specified mechanisms. A function name containing **private-proof** denotes an abstract snapshot event in the model. There is no cryptographic circuit, A2A server, ODRL compiler, external assessor integration or

general claim-validation engine. The additional plaintext reference gate implements only the restricted checks described in Appendix C. The manuscript’s propositions provide conditional mathematical arguments for the stated semantics. Reproducing the finite exploration does not discharge their implementation premises.

The model, results and manuscript are supplied for inspection and extension. A hash manifest records package file identities. The original standards review is dated 26 September 2026; the industry and case-study review is dated 27 September 2026. Mutable web specifications, issuer statuses and assessment scopes must be pinned or refreshed for any later implementation; dated recommendations and preprints are distinguished from developing proposals in the bibliography.

B.1 Inspecting a counterexample

The baseline checker separates local admission from remote progression. Its stored witnesses make this distinction reviewable. One valid trace is: issue a lease; construct a permitting snapshot; admit the operation; revoke authority; let the remote service accept the already admitted outbox operation; let it commit the exact effect. The resulting remote commitment is consistent with the model because revocation prevents new ordinary admission and does not erase an earlier admitted obligation. Whether dispatch itself can still be suppressed depends on the adapter and the selected transition; deployments should stop transmissions they still control.

The current-state ablation has a different trace: issue a lease; construct a permitting snapshot; revoke authority; admit using that snapshot. Its first violation occurs at the last step. The snapshot was genuine when produced, but the authoritative precondition no longer holds. A current-state comparison prevents this new admission. These traces should not be merged into a claim that all effects after revocation are either safe or forbidden.

For accounting, the valid cancellation witness issues, evaluates and admits, then requests cancellation, obtains remote confirmation and observes the authenticated terminal receipt. Closure releases the business reservation but retains the assurance debit. The refund ablation fails at closure because it removes that permanent charge. The protected account represents issuance of supported continuations, not the amount of unspent business resource.

B.2 Empirical reporting and reproduction limits

The search counts unique finite states and enabled successor edges; it does not sample trajectories. Breadth-first discovery supplies shortest ablation traces under the encoded event relation. Different abstractions, bounds or symmetry reductions would change these counts. All nine predicates are checked after every generated transition in the unmodified baseline. The report also retains event counts and positive witnesses, making vacuous refusal distinguishable from useful admitted behaviour.

No random seed, confidence interval or statistical significance test applies to this exhaustive finite enumeration. The state bounds, abstract proof events, diagnostic pending-coverage flag and enabled remote transitions define its scope. The separate reference gate in Appendix C tests concrete functions on selected fixtures; its scenario count is not additional exhaustive-state coverage. Reproduction of both artifacts is necessary for the reported software results, but insufficient for a deployment safety claim.

C Executable reference gate and functional experiments

C.1 Artifact and implementation scope

The revision adds a dependency-free Python reference implementation of selected evidence and execution semantics. This is a second executable artifact, separate from the finite transition model in Section 6. It supplements the abstract truth-table illustrations with functions that consume typed claims, source records, recognition paths, actions and lease plans. Its purpose is to make the protocol’s guards inspectable and reproducible. It is neither an OpenClaw integration nor an implementation of the private-proof relation.

The source in the package’s `reference_gate` directory consists of `reference_gate.py` and `run_scenarios.py`. The first implements the evaluator and protected-gate abstraction. The second supplies synthetic positive and adversarial fixtures, invokes those functions and checks both returned results and state changes. Running `python3 run_scenarios.py` prints individual outcomes, writes `scenario_results.json` and returns a failing exit code if a scenario fails. The recorded execution used Python 3.12.14 on 27 September 2026. All 29 scenarios passed their stated assertions. The identifiers RG01–RG29 are stable references to the scenarios in source order.

C.2 Implemented contracts

The reference uses two required evidence partitions: assessments and adverse records. Each registry supplies a complete sorted finite sequence. Its content identity commits to the source identifier, revision and all records. A gate-issued challenge fixes both partition roots. Omitting a required partition yields Indeterminate. Removing an existing adverse record from a presented sequence changes its root and yields Deny. An authenticated-in-the-model adverse record that matches the deployment and blocks reliance also yields Deny. A malformed adverse record yields Error, including when the missing field would otherwise conceal its blocking meaning.

Source authentication is a trusted-registry premise. The SHA-256 digest supplies content binding within the artifact; it does not verify a signature or establish issuer identity. Assessment fields identify the original assessor separately from an evidence wrapper. Validation matches the proposed claim to its report, assessment object, configuration, use context, method, result and validity interval. Recognition starts from a local root and checks each edge’s parent, scope, delegation depth, validity and status. Root and edge updates advance a trust revision used by admission. A wrapper cannot become the assessor by changing its issuer field.

The only implemented composition rule defines one explicit conjunction: named egress and tool-scope control predicates jointly support a correspondingly named control-completeness proposition. It supplies no inference from those controls to general system safety or regulatory compliance. Exact integer unit mapping accepts Wh and kWh and rejects unsupported units. These are synthetic business-resource quantities associated with the simulation request, rather than observed energy consumption.

The gate binds the exact action, actor, recipient, payload, configuration, context, persistent operation slot and episode. A decision also binds the evidence and policy versions, trust revision, time interval and lease context. Admission checks the clock-uncertainty interval against the accepted interval, compares current context, and records consumption, reservation, pending work and outbox state. Every guard executes before the admission writes in the serial reference execution.

Prospective evaluation constructs the exact hypothetical successor of a guarded lease transition. It does not charge the account. Lease issue checks the predecessor and recomputes the successor before appending the debit. Admission accepts the earlier prospective decision only when the actual episode context equals that successor. Renewal must include the actual pending-operation bindings. Closing a completed operation releases its reservation into recorded business expenditure, while its consumed slot and assurance debit remain.

C.3 Scenario design and observed results

Each scenario starts from a controlled fixture and changes a relevant condition or operation sequence. Positive fixtures establish that the implementation can admit and complete work. Adversarial fixtures assert the specific refusal reason and, where relevant, unchanged consumption, debit or pending state. The following groups describe the executed checks; the JSON output contains every individual result.

Table 14: Reference-gate functional scenarios and observed outcomes

Scenarios	Manipulation or sequence	Observed result
RG01	Issued lease, admission, receiver execution and terminal closure	One admission and commitment; debit retained; pending record closed
RG02–RG05	Missing adverse partition, omitted record, declared adverse finding, post-evaluation update	Indeterminate or Deny at the applicable boundary
RG06–RG07	Deployment changes or exact payload substitution	Admission refused
RG08–RG10	Wrapper substituted for assessor, identity-only competence, unrooted path	Claim reliance refused
RG11–RG12	Unsupported system-safety composition versus the approved control conjunction	Unsupported inference refused; defined conjunction admitted
RG13–RG15	Exact unit conversion, unknown unit, clock interval crossing expiry	Exact conversion admitted; unknown unit Error; expired interval refused
RG16–RG18	Exact, unrelated or forged prospective successor	Exact successor admitted; other successors refused
RG19–RG20	Reused operation slot or historical lease plan	Duplicate admission and stale transition refused
RG21–RG24	Renewal omission, valid renewal, cancellation request, actor replacement	Omitted pending work refused; legitimate changes retain debit and pending state
RG25	Duplicate delivery with and without receiver idempotency	One protected commitment versus two weak-receiver commitments
RG26–RG28	Recognition withdrawal, root expiry, malformed adverse record	Reliance refused or Error returned

Continued on next page

Table 14 (continued)

Scenarios	Manipulation or sequence	Observed result
RG29	Revocation, cancellation, closure, configuration/evidence changes and expiry before local dispatch	New local dispatch blocked by its applicable guard

RG16 observes zero debit after prospective evaluation and two abstract units after issue. RG22 observes three units after renewal while the earlier operation remains pending. RG20 retains five units after rejection of a replayed older lease plan. These values test accounting behaviour; they are not estimated harm probabilities.

RG25 deliberately weakens the receiver. The same locally admitted operation is delivered twice. The protected receiver commits once, while the weak receiver commits twice. Passing this scenario means that the harness detects the expected difference. It does not mean that the weak variant satisfies the at-most-once requirement.

C.4 Interpretation and limits

The reference adds concrete execution evidence for typed claim checks, complete declared views and prospective successor binding. It does not prove all properties of the code or exhaust its possible inputs. The 29 scenarios are deterministic functional experiments, not additional states in the 16,020-state exploration. The original model and its counts remain unchanged.

The evaluator returns the first applicable refusal. Its four-outcome interface therefore implements fail-closed decisions, rather than the full aggregation and global error-precedence rules of a general policy language. The accepted grammar is a closed set of structured Python records. There is no RDF processing, VC verification, ODRL compiler, zero-knowledge backend or cryptographic protocol execution. Actor and resource identities are declared fixture values; a live possession-key authentication protocol remains outside the artifact.

The controller runs in one process with protected-state access assumed. It demonstrates serial guarded mutations and rejection of stale transition replays. It does not implement crash-safe durability, a persistent anti-rollback mechanism, concurrent transactions or distributed agreement. A production implementation must substantiate those properties independently.

The adapter chooses the stricter local dispatch branch: authority, configuration, evidence, trust, interval and coverage are rechecked while an effect remains locally controllable. A refused dispatch retains pending obligations. The mock receiver immediately returns a completed result, so these experiments do not establish delayed remote cancellation, physical containment or deletion of copied data. Source truth, assessor competence in practice, legal effect, deployed behavioural quality and calibrated risk remain external validation tasks. No external model provider, OpenClaw instance or operational resource was tested, and no throughput or field-performance claim follows from these results.

D Provider-reported operational evidence

The table separates operational observations from the inference they support. These are provider reports, not experiments conducted for this paper. They ground the engineering guidance in published experience while preserving the distinction between deployed mechanisms and our proposed cross-organisation protocol.

Table 15: Published operational observations and their limits

Observation	Reported dataset or exposure	Reported result	Interpretation and limit
Anthropic sandbox [88]	Internal use; sample size and exposure duration unreported.	Permission prompts reduced by 84%.	Enforced boundaries can reduce repeated interaction. This does not establish an 84% reduction in harm or successful attacks.
Auto-mode full pipeline: ordinary traffic [90]	10,000 internal employee tool calls.	0.4% false-positive rate.	This evaluation measured unnecessary blocking. This does not establish universal operational overhead.
Auto-mode full pipeline: overreager actions [90]	52 curated real actions.	17% false-negative rate.	The evaluated classifier retained consequential misses. This does not establish deployment-wide harm probability.
Auto-mode full pipeline: exfiltration [90]	1,000 synthetic cases.	5.7% false-negative rate.	This test exposed residual errors. This does not establish general resistance to adaptive attackers.
OpenAI Atlas update [86]	Automated adversarial testing; no quantitative denominator for the reported update.	Attacks informed a deployed model update and surrounding safeguards.	Attack discovery was connected to remediation. This does not establish a numerical reduction in incident risk.

Numerical treatment. Preserve the rates exactly as published. The listed numbers are dataset sizes, not reconstructed confusion matrices. A false-positive rate uses benign examples as its denominator; a false-negative rate uses positive examples. The report does not supply all raw counts or sufficient detail to reconstruct those calculations. Rounding prevents reliable recovery of integer error counts. Do not infer, for example, exactly nine misses from 17% of 52. Do not pool the three datasets, derive confidence intervals from rounded rates alone, or use them as calibrated inputs to Paper 2.

Use in this paper. The corresponding engineering requirement is to repeat representative evaluations on the installed configuration and preserve labels, versions, assumptions and raw results. Paper 3 can authenticate and scope the resulting evidence. Its finite-state checker examines control invariants within a declared abstraction. These two forms of evidence answer different questions; an end-to-end deployment trial remains necessary.

E Extended proof obligations for a resource adapter

The five propositions in Section 4 concern a specified machine. This appendix identifies the steps a code-level refinement must discharge for the OpenClaw case. It is an expanded proof plan, not a proof of the runtime or of the reference Python implementation.

E.1 Evidence-view preservation

Take two executions with the same accepted profile, source commitments, query parameters and explicit context. A deterministic parser and evaluator must return the same typed result and dependencies. For Proposition 1, the witness must contain every record in each declared source partition, or an authenticated query proof must establish the same selected result. Length and root checks prevent a positive subset from masquerading as the declared source sequence. They cannot prevent the source owner from omitting an event before commitment.

A negative query requires a dependency even when its result is empty. Suppose the current partition contains no blocking incident. After the source adds a matching incident, the old record set remains an authentic historical view. Admission may rely on it only if the declared freshness rule still permits that view. If the rule requires the current partition version, inserting the incident must invalidate the decision. An implementation that records dependencies only on returned positive records cannot establish this step. Versioning the partition or proving a current authenticated query is necessary.

E.2 Recognition and semantic scope

For Proposition 2, each recognition edge must be authenticated, active and permitted by its parent. The effective scope is the intersection of all path scopes, including claim class, assessment object, method, purpose, jurisdiction and validity where relevant. Starting from an accepted root, induction over a bounded acyclic path shows that a delegated scope cannot exceed its parent's scope. A root's own validity and competence constraints are part of the base case; checking only downstream credentials leaves the induction unsound.

The same reasoning applies to semantic maps. A shape validator checks structure; it does not prove that a management-system predicate entails a deployed-system safety predicate. Every permitted map needs an explicit implication under its declared assumptions. Every composition rule needs the premises covering interactions and common dependencies. If those implications are unproved, a conforming evaluator must preserve the narrower claim or return an unsupported outcome. It cannot infer a broader proposition from the presence of several authentic reports.

E.3 Admission and physical object binding

For Proposition 3, locate the admission linearisation point: the committed transition where current dependencies, unconsumed operation identity, business-resource availability and the already issued covering continuation are checked together. A prospective proof additionally requires the authenticated issuance receipt and its matching successor context. A rejected admission must leave consumption, reservations, pending records and dispatchable outbox state unchanged. A separately successful lease issuance retains its full debit even if the subsequent admission is refused; failed issuance itself must leave no partial ledger mutation. A retry after an uncertain response reconciles the original issuance or operation instead of creating another entitlement. Transactions split across independently failing stores require an explicit coordination and recovery argument.

The adapter must also show that the object checked is the object used. A path may resolve through a changed symlink; a hostname may reach another tenant; a tool may rewrite parameters after approval. Bind stable object identifiers and the final request representation, or check the relation again at the last protected boundary. A hash of an earlier textual proposal does not establish that relation by itself. The projection in Proposition 5 must include every resulting external effect, including effects from subprocesses and recovery workers.

E.4 Accounting, recovery and delayed closure

For Proposition 4, induction over issue and renewal adds each accepted charge exactly once to the episode ledger. Other transitions leave the accumulated charge unchanged. Closing an operation may release its business reservation; replacing a worker changes its membership binding; neither subtracts a charge. Renewal carries every pending obligation into the successor context before acceptance. A stale proof for a different successor cannot justify that installation.

Recovery must preserve consumed slots, the append-only charge history, pending effects and outbox identity under crash and rollback. A receiver must independently deduplicate an operation or expose a reconciliation method with equivalent stated semantics. Local uniqueness does not establish remote uniqueness. A terminal receipt closes only the operation and outcome that it authenticates; an absence of response closes nothing. These obligations explain why atomic local admission, durable persistence and remote idempotence are separate implementation claims.

The resulting simulation proof establishes control invariants only for the declared effect family. Probability transfer additionally requires the stochastic conditions in Section 4.6, including timing, observations and private selection information. A successful test suite can expose counterexamples to these obligations; it cannot replace the universal arguments required for a full implementation proof.

References

The original standards and assessment-service review date is 26 September 2026. The industry and case-study sources were reviewed on 27 September 2026. Project manuscripts and developing specifications are identified as such.

- [1] Stöcker, C. (2026). Resource Bounded Program Equilibria under Cryptographic Commitments and Revocable Authority. Companion Paper 1, working manuscript, series revision 28 September 2026. Author’s research catalogue: <https://spherity.github.io/spherity-research/>. Companion PDF (same folder).
- [2] Stöcker, C. (2026). Risk-Bounded Runtime Assurance for Dynamic Multi-Agent Systems under Partial Observation. Companion Paper 2, working manuscript, series revision 28 September 2026. Author’s research catalogue: <https://spherity.github.io/spherity-research/>. Companion PDF (same folder).
- [3] Bauer, L.; Schneider, M. A.; and Felten, E. W. *A Proof-Carrying Authorization System*. Princeton University Technical Report TR-638-01, 30 April 2001. <https://www.cs.princeton.edu/techreports/2001/638.pdf>.
- [4] Ruess, H., and Shankar, N. *Evidential Transactions with Cyberlogic*. SRI International, CSL Technical Report SRI-CSL-2023-01, 20 March 2023. <https://arxiv.org/abs/2304.00060v1>. DOI: <https://doi.org/10.48550/arXiv.2304.00060>.
- [5] Schanzenbach, M.; Kilian, T.; Schütte, J.; and Banse, C. “ZKclaims: Privacy-preserving Attribute-based Credentials using Non-interactive Zero-knowledge Techniques.” *Proceedings of the 16th International Joint Conference on e-Business and Telecommunications, Volume 2: SECRIPT*, pp. 325–332, 2019. DOI: <https://doi.org/10.5220/0007772903250332>. Author preprint: <https://arxiv.org/abs/1907.09579v1>.
- [6] Rosenberg, M.; White, J.; Garman, C.; and Miers, I. “zk-creds: Flexible Anonymous Credentials from zkSNARKs and Existing Identity Infrastructure.” *IEEE Symposium on Security and Privacy*, 2023. Author manuscript: <https://www.cs.purdue.edu/homes/white570/media/zk-creds.pdf>.
- [7] Andersen, M. P.; Kumar, S.; AbdelBaky, M.; Fierro, G.; Kolb, J.; Kim, H.-S.; Culler, D. E.; and Popa, R. A. “WAVE: A Decentralized Authorization Framework with Transitive Delegation.” *28th USENIX Security Symposium*, pp. 1375–1392, August 2019. <https://www.usenix.org/conference/usenixsecurity19/presentation/andersen>.
- [8] Kryptos Chalkias, K.; Maram, D.; Roy, A.; Wang, J.; and Yadav, A. (2025). Zero-Knowledge Authenticator for Blockchain: Policy-Private and Obliviously Updateable. In 7th Conference on Advances in Financial Technologies (AFT 2025). Leibniz International Proceedings in Informatics, vol. 354, pp. 2:1–2:23. <https://doi.org/10.4230/LIPIcs.AFT.2025.2>.
- [9] *ERC-8354: Confidential Agent Policy Verdicts*. Ethereum Improvement Proposals, proposal, accessed 26 September 2026. <https://ercs.ethereum.org/ERCS/erc-8354>.
- [10] ZKM Project. *Zero-Knowledge Mandates (ZKM) Protocol Specification*. Version 0.1.0-draft, revision 12 (history dated 8 September 2026); header dated 4 August 2026; accessed 26 September 2026. <https://zkm.dev/spec/>.
- [11] Pang, H. H.; Jain, A.; Ramamritham, K.; and Tan, K.-L. “Verifying Completeness of Relational Query Results in Data Publishing.” *Proceedings of ACM SIGMOD 2005*, pp. 407–418, 2005. DOI: <https://doi.org/10.1145/1066157.1066204>. Primary institutional record: https://ink.library.smu.edu.sg/sis_research/1136/.

- [12] W3C. Verifiable Credentials Data Model v2.0. Recommendation, 15 May 2025. Specification: <https://www.w3.org/TR/vc-data-model-2.0/>.
- [13] W3C. Bitstring Status List v1.0. Recommendation, 15 May 2025. Specification: <https://www.w3.org/TR/vc-bitstring-status-list/>.
- [14] W3C. Shapes Constraint Language. Recommendation, 20 July 2017. Specification: <https://www.w3.org/TR/shacl/>.
- [15] W3C. PROV-O The PROV Ontology. Recommendation, 30 April 2013. Specification: <https://www.w3.org/TR/prov-o/>.
- [16] W3C. RDF Dataset Canonicalization. Recommendation, 21 May 2024. Specification: <https://www.w3.org/TR/rdf-canon/>.
- [17] W3C. Decentralized Identifiers Core v1.0. Recommendation, 19 July 2022. Dated specification: <https://www.w3.org/TR/2022/REC-did-core-20220719/>.
- [18] Bernstein, G., and Sporny, M., eds. *Data Integrity BBS Cryptosuites v1.0: Achieving Unlinkable Data Integrity with Pairing-based Cryptography*. W3C Candidate Recommendation Draft, 10 September 2026. <https://www.w3.org/TR/2026/CRD-vc-di-bbs-20260910/>.
- [19] W3C. *ODRL Information Model 2.2*. W3C Recommendation, 15 February 2018. <https://www.w3.org/TR/odrl-model/>.
- [20] W3C. ODRL Vocabulary and Expression 2.2. Recommendation, 15 February 2018. Specification: <https://www.w3.org/TR/odrl-vocab/>.
- [21] ODRL Community Group. *ODRL Formal Semantics*. Developing community document, accessed 26 September 2026. <https://w3c.github.io/odrl/formal-semantics/>.
- [22] Mustafa, D.; Collarana, D.; Peng, Y.; Haque, R.; Lange-Bever, C.; Quix, C.; and Decker, S. *Denotational Semantics for ODRL: Knowledge-Based Constraint Conflict Detection*. Working draft, arXiv:2602.19883v1, 23 February 2026. <https://arxiv.org/abs/2602.19883v1>. DOI: <https://doi.org/10.48550/arXiv.2602.19883>.
- [23] Mustafa, D.; Collarana, D.; Kirrane, S.; Lange, C.; Quix, C.; Haque, R.; Peng, Y.; and Decker, S. *Axis-Aligned Semantics for ODRL: Resolving Dimensional Ambiguity in Policy Constraints*. Preprint, arXiv:2602.19878v3, 23 May 2026. <https://arxiv.org/abs/2602.19878v3>. DOI: <https://doi.org/10.48550/arXiv.2602.19878>.
- [24] Cutler, J. W.; Disselkoen, C.; Eline, A.; He, S.; Headley, K.; Hicks, M.; Hietala, K.; Ioannidis, E.; Kastner, J.; Mamat, A.; McAdams, D.; McCutchen, M.; Rungta, N.; Torlak, E.; and Wells, A. *Cedar: A New Language for Expressive, Fast, Safe, and Analyzable Authorization (Extended Version)*. arXiv:2403.04651v2, 8 March 2024. <https://arxiv.org/abs/2403.04651v2>. DOI: <https://doi.org/10.48550/arXiv.2403.04651>.
- [25] A2A Project. *A2A Protocol Specification, version 1.0.0*. Release 12 March 2026. <https://a2a-protocol.org/v1.0.0/specification/>.
- [26] A2A Project. *A2A v1.0.1 release*. Released 28 May 2026; changelog dated 26 May 2026; commit 3303592588e388e62e0f69f701af531d2f4e3991. <https://github.com/a2aproject/A2A/releases/tag/v1.0.1>.
- [27] A2A Project. *A2A Protocol Specification*, current documentation, Section 7.6.4. Accessed 26 September 2026. <https://a2a-protocol.org/latest/specification/>.
- [28] A2A Project. *Extensions*. Current documentation. Accessed 26 September 2026. <https://a2a-protocol.org/latest/topics/extensions/>.

- [29] Model Context Protocol Project. *Authorization*. Specification revision 2026-07-28. <https://modelcontextprotocol.io/specification/2026-07-28/basic/authorization>.
- [30] Hedberg, R., ed.; Jones, M. B.; Solberg, A. Å.; Bradley, J.; De Marco, G.; and Dzhuvinov, V. *OpenID Federation 1.0*. Final specification, 17 February 2026. https://openid.net/specs/openid-federation-1_0-final.html.
- [31] SPIFFE. Federation specification. Specification: https://spiffe.io/docs/latest/spiffe-specs/spiffe_federation/.
- [32] AP2 Project. *Agentic Payment Protocol (v0.2)*. Current specification, accessed 26 September 2026. <https://ap2-protocol.org/ap2/specification/>.
- [33] ISO/IEC 42001:2023. Information technology Artificial intelligence Management system. Edition 1, December 2023. ISO record: <https://www.iso.org/standard/42001>.
- [34] BSI Group. BSI secures ANAB accreditation to certify ISO/IEC 42001. 10 March 2026. Official source: <https://www.bsigroup.com/en-GB/insights-and-media/media-centre/press-releases/2026/march/bsi-secures-anab-accreditation-to-certify-isoiec-42001/>.
- [35] DNV. DNV als akkreditierte Zertifizierungsstelle für KI Managementsysteme zugelassen. 17 December 2024. Official source: https://www.dnv.de/news/2024/DNV_akkreditiert_fuer_ISO_IEC42001/.
- [36] TÜV SÜD. EU AI Act assessment and voluntary Attestation of Conformity. Current service scope. Official source: <https://www.tuvsud.com/en/services/technical-advisory/eu-ai-act-assessment>.
- [37] UL Solutions. First certifications under the AI safety testing service for AI enabled products. 12 March 2026. Official source: <https://www.ul.com/news/ul-solutions-issues-first-certifications-under-ai-safety-testing-service-ai-enabled-products>.
- [38] UL Solutions. Assessing AI Safety of Qcells Data Center Energy Management System Control Component. Case study. Official source: <https://www.ul.com/resources/assessing-ai-safety-qcells-data-center-energy-management-system-control-component>.
- [39] ISO/IEC 42006:2025. Requirements for bodies providing audit and certification of artificial intelligence management systems. July 2025. Official source: <https://www.iso.org/standard/42006>.
- [40] DAKKS and UKAS. Joint technical bulletin on the use of AI in accredited conformity assessment. Announcement, 6 March 2026. Official source: <https://www.dakks.de/en/pressrelease/dakks-and-ukas-publish-joint-technical-bulletin-on-the-use-of-artificial-intelligence-ai.html>.
- [41] Diaconu, J. “Agent Identity and Task-Based Access Control: How A2A and AGNTCY Identity Work Together.” AGNTCY Blogs, 25 August 2026. <https://blogs.agntcy.org/technical/identity/2026/08/25/agent-identity-tbac-a2a-task-authorization.html>.
- [42] European Union. Regulation (EU) 2024/1689 laying down harmonised rules on artificial intelligence, especially Articles 16–27, 43, 47, 49, 72 and 73. Regulation of 13 June 2024. Official text: <https://eur-lex.europa.eu/eli/reg/2024/1689/oj/eng>.
- [43] European Commission. Governance and enforcement of the AI Act. Official policy page: <https://digital-strategy.ec.europa.eu/en/policies/ai-act-governance-and-enforcement>.
- [44] Tabassi. Artificial Intelligence Risk Management Framework AI RMF 1.0. NIST AI 100-1, January 2023. Official publication: <https://nvlpubs.nist.gov/nistpubs/ai/nist.ai.100-1.pdf>.

- [45] ISO/IEC 42005:2025. Information technology Artificial intelligence AI system impact assessment. Edition 1, May 2025. ISO record: <https://www.iso.org/standard/42005>.
- [46] GLEIF. vLEI Ecosystem Governance Framework. Version 4.0, 25 March 2026. Framework: <https://www.gleif.org/en/organizational-identity/become-a-vlei-issuer-qvi/vlei-ecosystem-governance-framework>.
- [47] Inspect AI. Log files and evaluation record schema. Current documentation. Official source: <https://inspect.aisi.org.uk/eval-logs.html>.
- [48] AI Verify Foundation. Project Moonshot. Current toolkit description. Official source: <https://aiverifyfoundation.sg/tools/moonshot/>.
- [49] MLCommons. AILuminate Safety. Benchmark scope and methodology overview. Official source: <https://mlcommons.org/ailuminate/safety/>.
- [50] CycloneDX. AI Models and Model Cards. Model inventory fields and examples. Official source: <https://www.cyclonedx.org/use-cases/ai-models-and-model-cards/>.
- [51] GitHub. Artifact attestations. Official security and provenance documentation. Official source: <https://docs.github.com/en/actions/concepts/security/artifact-attestations>.
- [52] IETF. RFC 9334 Remote ATtestation procedureS Architecture. Informational RFC, January 2023. Official source: <https://www.rfc-editor.org/rfc/rfc9334.html>.
- [53] Project Veraison. Attestation verification services. Official implementation repository. Official source: <https://github.com/veraison/services>.
- [54] European Commission. Notified bodies and the NANDO directory. Official explanation of roles and scope. Official source: https://single-market-economy.ec.europa.eu/single-market/goods/building-blocks/notified-bodies_en.
- [55] MLCommons. AILuminate Agentic Product Maturity Ladder. Emerging framework, current project description. Official source: <https://mlcommons.org/ailuminate/agentic/>.
- [56] MLCommons. AILuminate Global Assurance Program. Programme announcement, 19 February 2026. Official source: <https://mlcommons.org/2026/02/ailuminate-global-assurance/>.
- [57] OpenClaw, Security. <https://docs.openclaw.ai/gateway/security>. Accessed 27 September 2026.
- [58] OpenClaw, Multi-agent routing. <https://docs.openclaw.ai/concepts/multi-agent>. Accessed 27 September 2026.
- [59] OpenClaw, Multi-agent sandbox and tools. <https://docs.openclaw.ai/tools/multi-agent-sandbox-tools>. Accessed 27 September 2026.
- [60] OpenClaw, Sandboxing. <https://docs.openclaw.ai/gateway/sandboxing>. Accessed 27 September 2026.
- [61] OpenClaw, Exec approvals. <https://docs.openclaw.ai/tools/exec-approvals>. Accessed 27 September 2026.
- [62] OpenClaw, A2A. <https://docs.openclaw.ai/channels/a2a>. Accessed 27 September 2026.
- [63] OpenClaw, Plugin hooks. <https://docs.openclaw.ai/plugins/hooks>. Accessed 27 September 2026.

- [64] OpenClaw maintainer advisory GHSA-3mq7-q27j-mq7q, Exec approvals could outlive their reviewed working directory. 11 September 2026. <https://github.com/openclaw/openclaw/security/advisories/GHSA-3mq7-q27j-mq7q>. Accessed 27 September 2026.
- [65] OpenClaw maintainer advisory GHSA-7jfq-rmfm-29wp, File-transfer approvals could widen durable authority. 11 September 2026. <https://github.com/openclaw/openclaw/security/advisories/GHSA-7jfq-rmfm-29wp>. Accessed 27 September 2026.
- [66] OpenClaw maintainer advisory GHSA-vhwf-4x96-vqx2, skills-install-download can be redirected outside the tools root by rebinding the validated base path. 11 March 2026. <https://github.com/openclaw/openclaw/security/advisories/GHSA-vhwf-4x96-vqx2>. Accessed 27 September 2026.
- [67] OpenClaw, Formal verification (security models). <https://docs.openclaw.ai/security/formal-verification>. Accessed 27 September 2026.
- [68] NVIDIA. “Agentic AI in the Factory.” *NVIDIA Enterprise AI Factory Design Guide White Paper*. Last updated 27 May 2026. <https://docs.nvidia.com/ai-enterprise/planning-resource/ai-factory-white-paper/latest/agentic-ai-in-the-factory.html> Accessed 27 September 2026.
- [69] NVIDIA. *OpenShell v0.1.1*. GitHub release, 26 September 2026, release commit prefix 4ce767f. <https://github.com/NVIDIA/OpenShell/releases/tag/v0.1.1> Accessed 27 September 2026.
- [70] NVIDIA-NeMo. *Guardrails v0.24.1*. 16 September 2026. <https://github.com/NVIDIA-NeMo/Guardrails/releases/tag/v0.24.1> Accessed 27 September 2026.
- [71] NVIDIA. “Release Notes.” *NVIDIA NemoClaw User Guide*. Entry v0.0.129, 23 September 2026; early-preview availability began 16 March 2026. <https://docs.nvidia.com/nemoclaw/user-guide/openclaw/release-notes> Accessed 27 September 2026.
- [72] NVIDIA. “NemoClaw Enterprise Readiness and Admin Capability Guidance.” <https://docs.nvidia.com/nemoclaw/user-guide/openclaw/reference/enterprise-readiness> Accessed 27 September 2026.
- [73] NVIDIA. “Architecture.” *NVIDIA OpenShell*, documentation displaying v0.1.1. <https://docs.nvidia.com/openshell/latest/about/architecture> Accessed 27 September 2026.
- [74] NVIDIA. “Policy Prover.” *NVIDIA OpenShell*, documentation displaying v0.1.1. <https://docs.nvidia.com/openshell/latest/how-it-works/policies/prover> Accessed 27 September 2026.
- [75] NVIDIA. “Gateway Interceptors.” *NVIDIA OpenShell*, documentation displaying v0.1.1. <https://docs.nvidia.com/openshell/extensibility/gateway-interceptors> Accessed 27 September 2026.
- [76] NVIDIA. “Supervisor Middleware.” *NVIDIA OpenShell*, documentation displaying v0.1.1. <https://docs.nvidia.com/openshell/extensibility/supervisor-middleware> Accessed 27 September 2026.
- [77] NVIDIA. “Tool Calling.” *NVIDIA NeMo Guardrails Library Developer Guide*. <https://docs.nvidia.com/nemo/guardrails/configure-guardrails/guardrail-catalog/tool-calling> Accessed 27 September 2026.
- [78] NVIDIA. “AI Runtime Security FAQ.” *NVIDIA NeMo Guardrails Library Developer Guide*. <https://docs.nvidia.com/nemo/guardrails/resources/runtime-security-faq> Accessed 27 September 2026.

- [79] NVIDIA. “Verify Signed Agent Skills.” *NVIDIA Skill Documentation*. <https://docs.nvidia.com/skills/signing-agent-skills> Accessed 27 September 2026.
- [80] NVIDIA. “Evaluate Agent Skills Before Publication.” *NVIDIA Skill Documentation*. <https://docs.nvidia.com/skills/evaluating-agent-skills> Accessed 27 September 2026.
- [81] NVIDIA. “Trusted Computing Base for Lifecycle Control.” *NVIDIA NemoClaw User Guide*. <https://docs.nvidia.com/nemoclax/user-guide/openclaw/security/trusted-computing-base> Accessed 27 September 2026.
- [82] NVIDIA. “Safety for Agentic AI.” *NVIDIA Build*, developer-example catalogue entry. <https://build.nvidia.com/nvidia/safety-for-agentic-ai> Accessed 27 September 2026.
- [83] OpenAI, *Designing AI agents to resist prompt injection*, 11 March 2026. <https://openai.com/index/designing-agents-to-resist-prompt-injection/> Accessed 27 September 2026.
- [84] OpenAI, *Safety in building agents*, live documentation. <https://developers.openai.com/api/docs/guides/agent-builder-safety> Accessed 27 September 2026.
- [85] OpenAI, *Sandbox Agents*, live documentation. <https://developers.openai.com/api/docs/guides/agents/sandboxes> Accessed 27 September 2026.
- [86] OpenAI, *Continuously hardening ChatGPT Atlas against prompt injection attacks*, 22 December 2025. <https://openai.com/index/hardening-atlas-against-prompt-injection/> Accessed 27 September 2026.
- [87] Anthropic, *Building effective agents*, 19 December 2024. <https://www.anthropic.com/engineering/building-effective-agents> Accessed 27 September 2026.
- [88] Anthropic, *Beyond permission prompts: making Claude Code more secure and autonomous*, 20 October 2025. <https://www.anthropic.com/engineering/claude-code-sandboxing> Accessed 27 September 2026.
- [89] Anthropic, *How we contain Claude across products*, 25 May 2026. <https://www.anthropic.com/engineering/how-we-contain-claude> Accessed 27 September 2026.
- [90] Anthropic, *How we built Claude Code auto mode: a safer way to skip permissions*, 25 March 2026. <https://www.anthropic.com/engineering/claude-code-auto-mode> Accessed 27 September 2026.
- [91] OpenAI, *OpenAI’s Frontier Governance Framework*, 28 May 2026. <https://openai.com/index/openai-frontier-governance-framework/> Accessed 27 September 2026.
- [92] Anthropic, *Responsible Scaling Policy*, version 3.0, effective 24 February 2026. <https://anthropic.com/responsible-scaling-policy/rsp-v3-0> Accessed 27 September 2026.
- [93] OpenAI, *Priorities and principles for effective third party assessments*, 22 September 2026. <https://openai.com/index/priorities-principles-third-party-assessments/> Accessed 27 September 2026.
- [94] OpenAI, *Our framework for reporting model misalignment*, 16 September 2026. <https://openai.com/index/model-misalignment-reporting-framework/> Accessed 27 September 2026.